



Middle East University
Faculty of Information Technology

AI Food Recognition System

A project submitted in partial fulfillment of the requirements for the degree of
Artificial Intelligence / Computer Science

by:

Mohannad Alhemsi
(Student ID: 202210143)

Shatha Alghrair
(Student ID: 202220317)

Supervised By:
Maria Yousef

June, 2026

Declaration of Originality and Academic Integrity

We, the undersigned, hereby declare that the work presented in this report is entirely our own, produced independently under the supervision of our designated supervisor. All ideas, text, data, figures, tables, and other content drawn from external sources have been clearly identified in the text and fully attributed in the reference list at the end of this document.

We confirm that this report has not been submitted in whole or in part to any other institution or for any other qualification. We are aware that any form of academic dishonesty — including plagiarism, data fabrication, unauthorized use of artificial intelligence generation tools without proper disclosure, or misrepresentation of contributions — constitutes a violation of the university's academic integrity policy and may be subject to disciplinary action.

We further undertake to notify the Supervisor and the Graduation Projects Committee of any material change to the project scope, team composition, or methodology within two weeks of such a change occurring.

#	Student Name	Student ID	Date	Signature
1	Mohannad Alhemsi	202210143		
2	Shatha Alghrair	202220317		

SUPERVISOR APPROVAL FOR SUBMISSION

I certify that this Graduation Project report, entitled:

"AI Food Recognition System"

prepared by the team listed above, has met the required academic standard for submission in partial fulfilment of the requirements for the degree of Bachelor of Science in Artificial Intelligence / Computer Science.

I confirm that the work presented is authentic, satisfies the project objectives approved at registration, and is ready for examination by the Graduation Projects Committee.

Supervisor Name: Maria Yousef

Dedication

We dedicate this work...
First and foremost, to Almighty Allah, for His boundless grace, countless blessings, and divine guidance. It is only by His favor and mercy that we found the strength to persevere, overcome every obstacle, and successfully reach this milestone. Alhamdulillah for His endless blessings.

To our beloved families, whose unconditional love, endless sacrifices, and continuous prayers have been the bedrock of our journey and the light guiding us through every challenge.

To our friends, for their unwavering encouragement, shared memories, and for being a constant source of strength and motivation when it mattered most.

To our distinguished professors and Middle East University, for imparting knowledge, fostering our intellectual growth, and inspiring us to pursue excellence in the field of Artificial Intelligence.

Mohannad & Shatha

Acknowledgements

First and foremost, we thank Almighty Allah for granting us the strength, patience, and perseverance to complete this work.

We would like to express our deepest gratitude to our supervisor, Maria Yousef, for her invaluable guidance, continuous feedback, and academic insight throughout the duration of this project. Her expertise and encouragement have been instrumental in shaping the direction and quality of this research.

We are also grateful to the Faculty of Information Technology at Middle East University (MEU) for providing the necessary resources and a supportive academic environment. Special thanks go to the Graduation Projects Committee for their evaluation and constructive feedback.

Our sincere appreciation extends to the developers and researchers behind the open-source tools and datasets used in this project, including the Food-101 dataset, the USDA FoodData Central API, PyTorch, React Native, and PostgreSQL. Their contributions to the scientific and technical community made this work feasible.

Finally, we thank our families and friends for their unwavering moral support and understanding during the long hours of research and development. Their encouragement has been a constant source of motivation.

Contents

Declaration of Originality and Academic Integrity **1**

Dedication **2**

Acknowledgements **3**

Abbreviations **9**

Abstract **11**

1 Introduction **13**
1.1 Background and Context 13
1.2 Problem Statement 14
1.3 Proposed Solution 14
1.4 Project Objectives 15
1.5 Project Scope 15
1.6 Beneficiaries and Stakeholders 16
1.7 Report Organisation 16

2 Project Plan **18**
2.1 Project Deliverables 18
2.2 Project Tasks 19
2.2.1 Phase 1 Timeline (Gantt Chart) 19
2.2.2 Task Dependency Diagram (PERT) 20
2.3 Roles and Responsibilities 20
2.4 Risk Assessment 20
2.5 Cost Estimation 21
2.6 Project Management Tools 22

3 Literature Review and Related Work **23**
3.1 Related Systems and Prior Work 23
3.1.1 MyFitnessPal 23
3.1.2 FoodVisor 23
3.1.3 Lose It! 23
3.1.4 Cal AI 23
3.1.5 Im2Calories (Research Prototype) 24
3.1.6 Comparative Summary 24
3.2 Technology Review 24
3.2.1 Image Classification Architecture 24
3.2.2 Training Dataset 25

3.2.3 Nutritional Database . . . 25
3.2.4 Deep Learning Framework . . . 26
3.2.5 Application Platform . . . 26
3.3 Knowledge Gap . . . 27
3.4 Summary . . . 27

4 Requirements Specification 28
4.1 Stakeholders Analysis . . . 28
4.2 Platform Requirements . . . 29
4.2.1 Client-Side Requirements . . . 29
4.2.2 Server-Side Requirements . . . 29
4.3 Functional Requirements . . . 30
4.4 Non-Functional Requirements . . . 31
4.5 Other Requirements . . . 32

5 Architecture and Design 33
5.1 System Architecture . . . 33
5.2 Data Design . . . 34
5.2.1 Database Schema (3NF) . . . 35
5.3 Workflow and Process Design . . . 37
5.3.1 Use Case Diagram . . . 37
5.3.2 Sequence Diagram . . . 38
5.3.3 Activity Diagram . . . 39
5.3.4 Component Diagram . . . 40
5.3.5 Class and Module Overview . . . 41
5.3.6 Recommendation Engine Design . . . 43
5.4 Interface Design . . . 43
5.5 Technology Stack . . . 44
5.6 Development and Deployment Environment . . . 46
5.6.1 Development Environment . . . 46
5.6.2 Version Control Strategy . . . 46
5.6.3 Deployment Target . . . 46

6 Implementation 47
6.1 General Implementation Description . . . 47
6.1.1 Tools and Technologies Used . . . 47
6.1.2 The Machine Learning Models . . . 49
6.1.3 Environment and Execution . . . 51
6.2 Pipeline Implementation Description . . . 51
6.3 Model Implementation . . . 53
6.4 Additional Implementation Details . . . 60

7 Testing and Validation 69
7.1 Testing Strategy and Objectives . . . 69
7.2 Automated Backend Testing . . . 70
7.3 Machine Learning Model Evaluation . . . 72
7.4 Manual End-to-End Testing (User Acceptance Testing) . . . 74
7.5 Requirements Traceability and Test Summary . . . 81

8 Conclusions and Future Work **85**
8.1 Summary of Achievements . . . 85
8.2 Evaluation of Project Goals . . . 85
8.3 Limitations . . . 87
8.4 Future Work . . . 87
8.5 Lessons Learned . . . 89
8.5.1 Technical Challenges . . . 89
8.5.2 Project Management Challenges . . . 89

References **91**

A Users' Manual **93**
A.1 Installation and Configuration (Developer Setup) . . . 93
A.2 Using the Application . . . 94
A.3 Error Messages and Troubleshooting . . . 98

B Document Changes **100**

C Code Documentation **102**

D Deployment Guide **104**
D.1 Infrastructure Overview . . . 104
D.2 Configuration . . . 104
D.3 Startup Sequence . . . 105
D.4 Infrastructure Requirements for a Hosted Deployment . . . 105

E AI Tool Usage Disclosure **106**

List of Figures

5.1 High-Level System Architecture Diagram . . . 33
5.2 Entity-Relationship Diagram — AI Food Recognition System . . . 34
5.3 Use Case Diagram — AI Food Recognition System . . . 38
5.4 Sequence Diagram — Food Recognition and Nutritional Estimation Flow 39
5.5 Activity Diagram — End-to-End User Workflow . . . 40
5.6 Component Diagram — AI Food Recognition System . . . 41
5.7 Interface Wireframes — Key Screens . . . 43
5.8 Interface Wireframes — Additional Screens . . . 44

6.1 End-to-End Recognition and Logging Pipeline . . . 52
6.2 Login Screen . . . 61
6.3 Registration (Create Account) Screen . . . 62
6.4 Home Dashboard . . . 63
6.5 Camera Capture Screen . . . 64
6.6 Meal History Screen . . . 65

6.7 Daily Insights Screen — Unlocked View 66
6.8 Daily Insights Screen — Additional Insights and Disclaimer 67
6.9 Profile Screen 68

7.1 Test Case 1 — Recognition Result (Hamburger, 87%) 75
7.2 Test Case 1 — Nutrition Summary (1,584 kcal) 76
7.3 Test Case 2 — Recognition Result (Omelette, 85%) 77
7.4 Test Case 2 — Nutrition Summary, Served from Cache (349 kcal) 78
7.5 Test Case 3 — Recognition Result on a Real Device Photo (Deviled Eggs, 75%) 79
7.6 Test Case 4 — Recognition Result at the Confidence Boundary (Hamburger, 60%) 80
7.7 Test Case 4 — Nutrition Summary (746 kcal, Fried, Cached USDA) . . . 80

A.1 Create an account with an email, username, password, and initial health goal. 94
A.2 Returning users log in with their email and password. 95
A.3 The Home dashboard shows today's calories, macros, and recent meals. Tap “Log a meal” to start the capture flow. 95
A.4 Take a photo with the shutter button, or tap “Gallery” to choose an existing photo. 96
A.5 Confirm the AI's prediction (or choose an alternative if unsure), then adjust the estimated portion if needed. 96
A.6 Select how the food was cooked and review the nutrition breakdown before logging the meal. 97
A.7 The History tab lists every logged meal, newest first, with a Delete option on each row. 97
A.8 The Profile tab lets a user change their health goal, update body metrics, or log out. 98

List of Tables

1 List of Abbreviations 9

2.1 Project Deliverables 18
2.2 Phase 1 Task Breakdown 19
2.3 Phase 1 Gantt Chart (14 Weeks) 19
2.4 Roles and Responsibilities 20
2.5 Risk Assessment 21
2.6 Cost Estimation 21

3.1 Comparison of Related Dietary Monitoring Systems 24
3.2 Comparison of Candidate Classification Architectures 25
3.3 Comparison of Candidate Food Image Datasets 25

3.4 Comparison of Candidate Nutritional Data Sources 26
3.5 Comparison of Deep Learning Frameworks 26

4.1 Stakeholder Analysis 28
4.2 Client-Side Platform Requirements 29
4.3 Server-Side Platform Requirements 29
4.4 Functional Requirements 30

5.1 Database Schema — Third Normal Form 35
5.2 Module / Class Overview — Backend Processing Layer 41
5.3 Technology Stack Justification 44

6.1 Implementation Tools and Technologies 47
6.1 Implementation Tools and Technologies 48
6.1 Implementation Tools and Technologies 49
6.2 Machine Learning Models in the Served Pipeline 50
6.3 Reference Objects and Their Real-World Dimensions 56
6.4 Container Size Priors 57

7.1 Testing Levels Applied to the AI Food Recognition System 69
7.2 Automated Backend Test Modules 70
7.2 Automated Backend Test Modules 71
7.2 Automated Backend Test Modules 72
7.3 Classification Model Evaluation Results 73
7.4 Portion Estimation Evaluation Results 74
7.5 Functional Requirements Traceability 81
7.5 Functional Requirements Traceability 82
7.6 Non-Functional Requirements Traceability 82
7.6 Non-Functional Requirements Traceability 83

8.1 Evaluation of Project Objectives 86
8.2 Proposed Future Enhancements 88

A.1 Development Prerequisites 93
A.2 Common Errors and Resolutions 98
A.2 Common Errors and Resolutions 99

B.1 Changes from Project 1 to Project 2 100
B.1 Changes from Project 1 to Project 2 101

C.1 Representative Code Documentation 102
C.1 Representative Code Documentation 103

D.1 Deployment Configuration Variables 104
D.1 Deployment Configuration Variables 105

E.1 AI Tool Identification 106
E.2 AI Tool Usage by Task 106
E.2 AI Tool Usage by Task 107
E.2 AI Tool Usage by Task 108

Abbreviations

Table 1: List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package Kit
CNN	Convolutional Neural Network
CV	Computer Vision
DB	Database
DL	Deep Learning
ERD	Entity-Relationship Diagram
GPU	Graphics Processing Unit
HTTPS	Hypertext Transfer Protocol Secure
IPA	iOS Package Archive
IoU	Intersection over Union
JSON	JavaScript Object Notation
JWT	JSON Web Token
LTS	Long-Term Support
MEU	Middle East University
ML	Machine Learning
MLP	Multi-Layer Perceptron
MVP	Minimum Viable Product
NFC	Near Field Communication
3NF	Third Normal Form
ORM	Object-Relational Mapping
PERT	Programme Evaluation and Review Technique

Continued on next page

Table 1 – continued from previous page

Abbreviation	**Meaning**
REST	Representational State Transfer
RGB	Red, Green, Blue
TLS	Transport Layer Security
UAT	User Acceptance Testing
UML	Unified Modelling Language
USDA	United States Department of Agriculture
VRAM	Video Random-Access Memory
WHO	World Health Organization

Abstract

Objective

This project addresses four critical deficiencies in existing dietary monitoring applications: the absence of automated portion size estimation from photographs, inadequate handling of multi-component dishes, low classification accuracy in deployed systems, and a lack of personalised dietary recommendations derived from individual meal history. The primary objective is to design and implement an AI Food Recognition System that allows users to photograph a meal and receive an automated nutritional breakdown together with personalised guidance.

Methodology

The solution is structured as a three-tier mobile application. The food recognition module is developed by fine-tuning an EfficientNet-B4 model on the Food-101 dataset using transfer learning. Nutritional data is sourced exclusively from the laboratory-verified USDA FoodData Central API. A multi-stage portion size estimation pipeline operates on standard two-dimensional images through container classification, reference-object calibration, and shape-prior inference. A content-based filtering recommendation engine aggregates user meal history to generate daily dietary insights aligned with user-defined health goals. The system is built with React Native (mobile frontend), FastAPI and PyTorch (backend and deep learning), and PostgreSQL (database).

Findings

Phase 1 produced a complete requirements specification (functional, non-functional, and platform requirements), a third-normal-form database schema comprising five core tables (user, meal_log, meal_item, food_item, recommendation), UML workflow diagrams (use case, sequence, activity, component), UI mockups for nine primary screens, and a justified technology stack. Phase 2 delivered a fully working, end-to-end mobile application: authentication, a live camera capture flow, AI-based food recognition (EfficientNet-B4 at 91.4% top-1 accuracy on Food-101), USDA-backed nutrition lookup, meal logging and history, a user profile with editable health goals and body metrics, and a personalised insights screen — all implemented, tested on physical devices, and version-controlled on GitHub.

Conclusion

The completed system establishes a clear differentiation from current commercial applications by directly addressing the identified knowledge gaps. The architecture is modular, testable, and scalable. The portion estimation pipeline produces calibrated range estimates without specialised hardware, while the recommendation engine provides personalised insights after a minimum of seven logged meals. The working prototype

demonstrates that all Phase 1 design goals were realised in a functioning, testable mobile application.

Scope

The food recognition component is limited to the 101 categories of Food-101. Nutritional data retrieval is restricted to the USDA API. Portion estimation operates on standard RGB images. Regional cuisines not represented in Food-101 (e.g., Arabic and Middle Eastern dishes) remain outside the current scope and are identified as a primary future extension. The recommendation engine requires a minimum meal history to generate reliable insights.

1 Introduction

1.1 Background and Context

The global prevalence of diet-related non-communicable diseases has emerged as one of the most pressing challenges in contemporary public health. According to the World Health Organization, obesity has nearly tripled worldwide since 1975, with more than 650 million adults classified as obese and over 1.9 billion adults classified as overweight. Unhealthy dietary patterns are a primary contributing factor to this epidemic, as they are closely associated with the onset of cardiovascular disease, type 2 diabetes mellitus, hypertension, and certain forms of cancer. In this context, the accurate monitoring of dietary intake has been widely recognised as a fundamental strategy for the prevention and management of these conditions.

Despite the established clinical importance of dietary assessment, traditional methods of food logging remain inadequate for widespread adoption in real-world settings. Self-reported dietary recalls, food diaries, and weighed food records are known to suffer from systematic inaccuracies arising from portion-size misestimation, retrospective recall bias, and poor long-term user compliance. These limitations have motivated researchers and engineers to explore computational approaches to dietary monitoring that can reduce the cognitive burden placed upon users while simultaneously improving the accuracy and objectivity of nutritional data collection.

The rapid advancement of deep learning and computer vision has provided an opportunity to address these limitations. Convolutional Neural Networks (CNNs) and Vision Transformers have demonstrated substantial capacity for image classification tasks, achieving high performance across a wide range of benchmark datasets. Within the domain of food computing, these advances have enabled the development of systems capable of automatically recognising food items from photographic images, thereby opening a practical pathway toward passive and effortless dietary monitoring.

The introduction of large-scale food image datasets has been a critical enabler of progress in this domain. The Food-101 dataset established a foundational benchmark comprising 101,000 images across 101 food categories and has since become a standard reference for evaluating food recognition models. Subsequent work has produced increasingly capable classification architectures, with models such as EfficientNet and Vision Transformers demonstrating the technical progress of automated food recognition as a field.

Despite these advances at the classification level, commercially deployed dietary monitoring applications continue to exhibit significant limitations. A peer-reviewed evaluation study published in 2024, which assessed the performance of seven AI-enabled food tracking applications, found that no application was capable of automatically estimating portion sizes from photographs, and that automatic energy estimations derived from image analysis were consistently inaccurate across all platforms tested. Furthermore, existing systems generally do not accommodate composite and mixed dishes, where a single plate contains multiple distinct food components each possessing differing nutritional profiles. These gaps highlight a substantial and unresolved engineering challenge that this project

seeks to address.

1.2 Problem Statement

Notwithstanding the progress achieved in food image recognition research, a fundamental disconnect persists between the classification performance demonstrated in research settings and the practical effectiveness of deployed dietary monitoring systems. This disconnect stems from several interrelated technical deficiencies.

Fine-grained visual similarity. Dishes that are visually closely related — such as grilled and baked preparations of the same ingredient, or visually similar grain-based dishes — often share global image-level features that can confuse standard single-label classifiers. Without mechanisms designed to capture discriminative local texture and surface characteristics, classification confidence scores become unreliable, and misclassifications propagate directly into erroneous nutritional estimates.

Inadequate handling of composite dishes. The prevalence of multi-component meals in real-world dietary contexts is poorly accommodated by single-label classification. A plate of pasta bolognese, for instance, contains pasta, minced beef, tomato sauce, and cheese, each with a distinct nutritional profile. Systems relying on a single predicted label either disregard the multi-component nature of such dishes or assign nutritional values based on an oversimplified representation.

Absence of automated portion size estimation. This capability remains absent from all deployed consumer applications. Without reliable portion size estimation, even a correctly identified food item yields a nutritional estimate of limited value, since caloric content scales directly with the mass of the consumed portion.

Absence of personalised dietary guidance. The majority of dietary monitoring applications function as passive tracking tools that record intake without generating individualised nutritional recommendations derived from the user's historical data.

This project proposes the development of an AI Food Recognition System that addresses each of these deficiencies through a structured, multi-module architecture combining computer vision, verified nutritional database integration, intelligent portion estimation, and machine learning-based personalised recommendation.

1.3 Proposed Solution

The proposed solution is a mobile-accessible AI Food Recognition System, named NutriLens, that allows users to photograph a meal and receive an automated nutritional breakdown along with personalised dietary guidance. The system removes the need for manual food logging by replacing it with a photograph-based identification and estimation workflow.

The system consists of three integrated modules. The first is a food recognition module that uses a deep learning classification model, fine-tuned on the Food-101 dataset, to identify the food items present in a submitted photograph. The second is a nutritional estimation module that retrieves verified macronutrient data — calories, protein, carbohydrates, and fat — from the USDA FoodData Central database for each identified food item, and incorporates a multi-stage portion size estimation pipeline that infers food quantity from the photograph using reference objects, container geometry, and food-class

shape priors, without requiring specialised hardware beyond a standard smartphone camera. The third is a recommendation module that applies content-based filtering to the user's accumulated meal history to generate personalised daily dietary insights aligned with user-defined health goals.

The system is implemented as a native mobile application using the React Native (Expo) framework, deployable on both Android and iOS platforms, backed by a FastAPI/PostgreSQL backend. All three modules are integrated into a single user-facing mobile interface that presents food identification results, estimated macronutrient values, and dietary recommendations in a clear and accessible format.

1.4 Project Objectives

The primary aim of this project is to design, develop, and evaluate an AI Food Recognition System that identifies food items from photographic images, estimates their nutritional content, and provides personalised dietary guidance based on individual user meal history. The following objectives guide the realisation of this aim:

1. To develop a food image classification module capable of identifying food items from photographic input by applying transfer learning on the Food-101 benchmark dataset.

2. To implement a classification framework that enables reliable identification of food items within a single image, supported by a confidence-gated fallback strategy for low-confidence predictions.

3. To design an image preprocessing pipeline that improves the robustness of the classification module across varied real-world photographic conditions, including low-light environments and variable colour temperature.

4. To develop a portion size estimation module that infers food quantity through a multi-stage pipeline combining container-type classification, reference-object spatial calibration, and food-class geometric shape priors, producing nutritional estimates expressed as ranges rather than precise single values.

5. To integrate the USDA FoodData Central database as the primary nutritional reference source, enabling per-item retrieval of verified calorie, protein, carbohydrate, and fat values.

6. To design and implement a content-based filtering recommendation module that constructs individual nutritional profiles from accumulated user meal history and generates daily dietary insights aligned with user-defined health goals.

1.5 Project Scope

The scope of this project is defined by the following boundaries. The food recognition component is trained and evaluated on the Food-101 dataset, encompassing 101 food categories representative of a broad range of commonly consumed dishes. Nutritional data retrieval is performed through programmatic queries to the USDA FoodData Central API. The portion size estimation module operates on standard two-dimensional RGB

images captured by a smartphone camera, without reliance on specialised depth-sensing hardware. The recommendation module employs content-based filtering applied to individual user meal history, generating personalised suggestions relative to user-defined health goals. The system is delivered as a native mobile application built with React Native (Expo), compatible with Android and iOS.

The following items are explicitly outside the scope of the current project. The classification model is limited to the 101 food categories represented in the Food-101 training corpus; food items outside this taxonomy are not expected to be reliably identified. Regional and traditional cuisines not adequately represented in the Food-101 dataset — including many Arabic and Middle Eastern dishes — are outside the current scope and are identified as a primary direction for future development. The performance of the recommendation module is also sensitive to the volume of accumulated meal history; reliable personalised insights require a minimum number of logged meals (seven) before the system can construct a sufficiently representative user profile.

1.6 Beneficiaries and Stakeholders

End Users (General Public). Individuals seeking to monitor their dietary intake for personal health, weight management, or fitness purposes constitute the primary user group. The system is designed to reduce the effort required for dietary self-monitoring and to provide more objective and consistent nutritional estimates than manual logging methods.

Healthcare Professionals. Dietitians, nutritionists, and general practitioners may benefit from the system as a tool to support patient dietary assessment and monitoring between clinical consultations.

Researchers and Academics. The system's architecture and evaluation results may contribute to ongoing research in food computing, applied computer vision, and nutritional informatics, particularly with respect to hardware-independent portion estimation and personalised dietary recommendation.

Middle East University (Institutional Stakeholder). As the sponsoring institution, MEU has a direct interest in the academic quality, technical rigour, and timely completion of the project. The Faculty of Information Technology and the Graduation Projects Committee are the primary institutional stakeholders responsible for evaluating and approving the submitted work.

1.7 Report Organisation

The remainder of this report is organised as follows. Chapter 2 presents the project plan, including a detailed breakdown of project tasks, a Gantt timeline, PERT representation, role assignments, risk assessment, cost estimation, and project management tools. Chapter 3 presents a review of related systems and prior work, a comparative evaluation of candidate technologies, an identification of the knowledge gap, and a summary of how the review informed design decisions. Chapter 4 presents the requirements specification for the proposed system, encompassing stakeholder analysis, platform requirements, functional requirements, non-functional requirements, and additional technical constraints. Chapter 5 presents the complete system architecture and design, covering the system

structure, data model, workflow diagrams, interface design, technology stack, and development environment.

2 Project Plan

2.1 Project Deliverables

The project is expected to produce the following deliverables upon completion. Each deliverable corresponds to a distinct phase of the development lifecycle and serves as a measurable output against which project progress is assessed.

Table 2.1: Project Deliverables

No.	Deliverable	Description
D1	Project Plan Report	Documentation of project tasks, timeline, roles, risks, and cost estimates (Phase 1 submission).
D2	Requirements Specification	A complete document describing all functional and non-functional system requirements, stakeholder analysis, and platform constraints.
D3	System Architecture Document	Formal system architecture diagram, data design, workflow diagrams, and interface mockups.
D4	Trained Classification Model	A fine-tuned deep learning model for food image recognition, trained on Food-101, with documented evaluation results.
D5	Portion Estimation Module	An implemented and evaluated multi-stage portion size estimation pipeline operating on standard smartphone images.
D6	Nutritional Integration Module	A working integration with the USDA FoodData Central API for per-item macronutrient retrieval.
D7	Recommendation Module	A content-based filtering engine that generates personalised dietary insights from accumulated user meal history.
D8	Integrated Application	A fully integrated native mobile application (React Native / Expo) combining all modules into a single user-facing interface.
D9	Test Report	Documented results of unit, integration, system, and user acceptance testing for all modules.
D10	Final Project Report	Complete project report submitted to MEU.

2.2 Project Tasks

Table 2.2 presents the high-level tasks for Phase 1 of the project, spanning 14 weeks and covering all planning, research, requirements, and design work.

Table 2.2: Phase 1 Task Breakdown

ID	Task Name	Description	Duration	Depends On
T1	Project Planning & Initiation	Define project scope, objectives, deliverables, and risks. Produce the project plan document.	2 weeks	None
T2	Literature Review & Related Work	Survey related systems and prior work. Compare candidate technologies and identify the knowledge gap.	4 weeks	T1
T3	Requirements Specification	Identify and document all functional and non-functional system requirements through stakeholder analysis.	3 weeks	T1, T2
T4	System Architecture & Design	Design the overall system architecture, data model, workflow diagrams, use case diagrams, and interface mockups.	4 weeks	T3
T5	Phase 1 Report Finalisation	Review, revise, and compile all Phase 1 chapters into the final submission document.	1 week	T4

2.2.1 Phase 1 Timeline (Gantt Chart)

Table 2.3 illustrates the planned week-by-week timeline for all Phase 1 tasks across the 14-week period.

Table 2.3: Phase 1 Gantt Chart (14 Weeks)

Task	Weeks Active
T1: Project Planning	W1–W2
T2: Literature Review	W2–W5
T3: Requirements Spec.	W5–W7
T4: Architecture & Design	W7–W10
T5: Report Finalisation	W13

2.2.2 Task Dependency Diagram (PERT)

The following sequence summarises the task dependencies for Phase 1. Tasks proceed sequentially, with a one-week overlap between T1/T2 and T2/T3 to maintain momentum.

$$\mathbf{T1} \rightarrow \mathbf{T2} \rightarrow \mathbf{T3} \rightarrow \mathbf{T4} \rightarrow \mathbf{T5}$$

The critical path for Phase 1 runs through all five tasks, with a total planned duration of 14 weeks. No tasks in Phase 1 proceed in parallel, as each task produces the inputs required by the next.

2.3 Roles and Responsibilities

This project is carried out by two students under academic supervision, Mohannad Alhemsi and Shatha Alghrair, who share joint responsibility for all aspects of project execution — requirements analysis, system design, model development and training, backend and mobile application implementation, database design, integration, testing, documentation, and deployment. The workload was divided collaboratively, with each contributing equally to research, development, and writing tasks.

The supervisor, Maria Yousef, provides academic oversight, constructive feedback, and formal approval at each project milestone.

Table 2.4: Roles and Responsibilities

Role	Person	Responsibilities
Student Developers	Mohannad Alhemsi, Shatha Alghrair	Jointly responsible for all analysis, design, implementation, testing, and documentation tasks across both phases. Produce all deliverables (D1–D10) together and maintain the project timeline.
Project Supervisor	Maria Yousef	Provides guidance on project direction and academic standards. Reviews and approves deliverables at each phase milestone. Evaluates the final submitted report.
Academic Evaluator	MEU Graduation Committee	Assesses the submitted report and final presentation against university graduation project standards.

2.4 Risk Assessment

Table 2.5 identifies the primary risks associated with the project tasks, together with their estimated probability, potential impact, and planned mitigation strategy.

Table 2.5: Risk Assessment

ID	Risk Description	Prob.	Impact	Mitigation Strategy
R1	The Food-101 dataset may not adequately represent certain dish categories, leading to lower-than-expected classification performance.	Medium	High	Apply data augmentation (rotation, brightness variation, cropping) to expand under-represented classes; monitor per-class performance during evaluation.
R2	Insufficient compute resources may delay or prevent training of the deep learning model within the project timeline.	Medium	High	Utilise Google Colab Pro and Kaggle GPU environments for training; schedule training runs during off-peak hours.
R3	The USDA FoodData Central API may be temporarily unavailable, disrupting nutritional data retrieval.	Low	Medium	Implement a local cache of frequently queried food entries; fall back to cached values when the API is unreachable.
R4	Portion size estimation may produce insufficiently accurate results due to the inherent ambiguity of two-dimensional images.	High	Medium	Produce calibrated range estimates rather than point values; prompt the user to confirm or adjust the estimated portion.
R5	Scope creep from attempting to include Arabic or Middle Eastern dishes not present in Food-101.	Medium	Medium	Strictly enforce the project scope boundary; document out-of-scope cuisines as future work.

2.5 Cost Estimation

The project was developed using primarily open-source tools and publicly available datasets. Table 2.6 summarises the estimated costs.

Table 2.6: Cost Estimation

Item	Description	Est. Cost (USD)	Category
Google Colab Pro	Cloud GPU compute for model training and experimentation (approx. 3–4 months of active development).	$40	Compute

Continued on next page

Table 2.6 – continued from previous page

Item	Description	Est. Cost (USD)	Category
Kaggle GPU Quota	Free-tier GPU access for supplementary training runs and experimentation.	$0	Compute
Food-101 Dataset	Publicly available dataset; no licensing cost.	$0	Data
USDA FoodData Central API	Free public API; no licensing cost.	$0	Data
Development Libraries	PyTorch, FastAPI, React Native, and related open-source libraries.	$0	Software

The total estimated cost of the project is $40, as the development stack relies entirely on open-source software, free public datasets and APIs, and affordable cloud compute resources.

2.6 Project Management Tools

Version Control (GitHub). All source code, model training scripts, and documentation are maintained in a private GitHub repository, using a branching strategy (main / develop / feature/*) that separates development, testing, and stable release versions.

Compute Environment (Google Colab Pro and Kaggle). Model training and experimentation were conducted using Google Colab Pro and Kaggle Notebooks, both providing GPU-accelerated compute environments supporting PyTorch.

Task Tracking (Trello). Project tasks and their completion status were tracked using a Trello board organised into columns corresponding to project phases (Backlog, In Progress, Under Review, Completed).

Documentation (LaTeX / Overleaf). All project documentation, including this report, is authored in LaTeX using the Middle East University graduation project template compiled on Overleaf.

Development Framework (Python / PyTorch / FastAPI / React Native). The backend is implemented in Python using PyTorch for model development and FastAPI for the REST API server, with SQLAlchemy/Alembic for the PostgreSQL data layer. The client-side mobile application is developed using React Native (Expo SDK), producing a single TypeScript codebase deployable on Android and iOS.

3 Literature Review and Related Work

3.1 Related Systems and Prior Work

This section reviews five existing systems that address dietary monitoring and food image recognition. Each system is evaluated with respect to its core features, technical approach, and limitations. The findings from this review directly inform the design decisions of the proposed system.

3.1.1 MyFitnessPal

MyFitnessPal is one of the most widely adopted dietary monitoring applications, with a database exceeding 14 million food entries. The platform primarily relies on user-driven text search and barcode scanning. A limited image-based food logging feature requires manual correction of identified items in most cases and does not support automated portion size estimation. The application does not support multi-component dish recognition, and personalised recommendations are limited to daily macronutrient targets derived from user-input goals rather than analysis of accumulated meal history.

3.1.2 FoodVisor

FoodVisor is a French application that employs CNN-based image classification to identify food items from photographs, supporting up to five items per image. Portion size estimation relies on plate detection and user-guided selection from predefined serving sizes. A peer-reviewed evaluation reported top-1 accuracy of approximately 46%, rising to approximately 71% for mixed dishes when the correct item appeared among the top suggestions. Nutritional data is sourced from the French CIQUAL database, limiting coverage of non-French food categories.

3.1.3 Lose It!

Lose It! incorporates an AI-powered food recognition feature ("Snap It") with reported top-1 accuracy of approximately 46% in independent evaluation. Portion size must be confirmed or adjusted manually. The application supports goal-based calorie budgeting but does not generate ML-based personalised recommendations from historical intake data.

3.1.4 Cal AI

Cal AI offers a streamlined photograph-to-calorie workflow using food classification combined with a predefined serving-size lookup; no automated portion estimation from image geometry is performed. The application has not been subject to rigorous peer-reviewed evaluation and does not support multi-component dish decomposition or personalised recommendations.

3.1.5 Im2Calories (Research Prototype)

Im2Calories is an academic research prototype developed at Google representing one of the earliest attempts at fully automated food diary construction. The system used semantic segmentation and monocular depth estimation to infer volume and mass via density priors. It demonstrated proof-of-concept viability under controlled conditions but reported substantial estimation error in real-world settings, primarily due to the inherent ambiguity of depth reconstruction from a single uncalibrated 2D image.

3.1.6 Comparative Summary

Table 3.1: Comparison of Related Dietary Monitoring Systems

System	**Image Rec.**	**Auto Portion**	**Multi-Comp.**	**Verified DB**	**Pers. Rec.**
MyFitnessPal	Limited	No	No	Partial	No
FoodVisor	Yes (46%)	No	Partial (5)	Yes (CIQUAL)	No
Lose It!	Yes (46%)	No	No	Yes (USDA)	No
Cal AI	Yes (unverified)	No	No	Partial	No
Im2Calories	Yes (research)	Partial (lab)	No	No	No
NutriLens (Proposed)	Yes	Yes	Yes	Yes (USDA)	Yes

The comparative analysis reveals that no existing deployed system simultaneously addresses automated portion size estimation, multi-component dish recognition, and personalised recommendation from individual meal history. These three unaddressed capabilities define the primary scope of the proposed system.

3.2 Technology Review

3.2.1 Image Classification Architecture

Four deep learning architectures were considered for the food image classification module: VGGNet, ResNet-50, EfficientNet-B4, and Vision Transformer (ViT).

Table 3.2: Comparison of Candidate Classification Architectures

Architecture	ImageNet Top-1	Params	Fine-tune Food-101	Key Limitation
VGGNet-16	74.5%	138M	Moderate	Very large parameter count; slow training
ResNet-50	76.1%	25M	High	Lower efficiency than EfficientNet at comparable accuracy
EfficientNet-B4	83.0%	19M	High	Requires careful hyperparameter tuning
ViT-Base	81.8%	86M	Low	Data-hungry; underperforms on small datasets

Selected: EfficientNet-B4. EfficientNet-B4 was selected as the classification backbone due to its favourable combination of high ImageNet accuracy, relatively low parameter count, and documented strong performance on the Food-101 benchmark when fine-tuned from ImageNet pre-trained weights. ViT was not selected due to its data-hungry pre-training requirements; ResNet-50 was retained as a comparison baseline during development.

3.2.2 Training Dataset

Table 3.3: Comparison of Candidate Food Image Datasets

Dataset	Classes	Images	Public	Key Limitation
Food-101	101	101,000	Yes	Limited to 101 categories; no regional coverage
ISIA Food-500	500	399,726	Yes	Larger but heavier compute requirement
UEC Food-100	100	14,361	Yes	Very small per-class sample count; primarily Japanese cuisine

Selected: Food-101. Food-101 was selected as the primary training and evaluation dataset. It is the most widely used benchmark in the food recognition literature, enabling direct comparison against published results, and its 101,000 images provided sufficient volume for fine-tuning within the available compute budget.

3.2.3 Nutritional Database

Table 3.4: Comparison of Candidate Nutritional Data Sources

Source	Free API	Lab-Verified	Key Limitation
USDA FoodData Central	Yes	Yes	English-language entries; limited regional food coverage
Nutritionix API	Partial	Partial	Free tier has rate limits; some entries are user-contributed
Open Food Facts	Yes	No	Crowdsourced data; variable accuracy and completeness

Selected: USDA FoodData Central. The USDA FoodData Central API provides laboratory-verified macronutrient values per 100 g, is freely accessible without rate-limit restrictions under the public API tier, and maintains separate entries per preparation method (e.g., chicken, broiled vs. chicken, fried), enabling cooking-method-sensitive nutritional queries.

3.2.4 Deep Learning Framework

Table 3.5: Comparison of Deep Learning Frameworks

Framework	Strengths	Weaknesses
PyTorch	Flexible dynamic computation graph; widely used in research; strong community support for EfficientNet fine-tuning via torchvision.	Deployment tooling requires additional setup compared to TensorFlow Serving.
TensorFlow / Keras	Strong deployment tooling (TensorFlow Lite for mobile); simpler high-level Keras API.	Less flexible for custom training loops; research community has shifted toward PyTorch.

Selected: PyTorch. PyTorch was selected due to its flexibility for custom training loop design, its strong community of published EfficientNet fine-tuning implementations, and its first-class support in Google Colab and Kaggle GPU environments.

3.2.5 Application Platform

The system is developed as a native mobile application using React Native (via the Expo SDK), enabling a single TypeScript codebase deployable on both Android and iOS. React Native was selected over Flutter (Dart-based) and Ionic (web-based) due to its native camera and gallery APIs, its large ecosystem of community libraries, and the team's existing familiarity with the JavaScript/TypeScript toolchain.

3.3 Knowledge Gap

Gap 1: Absence of Automated Portion Size Estimation. No commercially deployed dietary monitoring application reviewed in this study provides automated portion size estimation from photographic input without manual user specification. The proposed system addresses this through a multi-stage estimation pipeline combining container-type classification, reference-object spatial calibration, and food-class geometric shape priors.

Gap 2: Inadequate Treatment of Multi-Component Dishes. With the partial exception of FoodVisor, reviewed systems do not decompose composite dishes into constituent components for individual nutritional analysis.

Gap 3: Low Classification Accuracy in Deployed Systems. Independent evaluation reported top-1 accuracy of approximately 46% for both FoodVisor and Lose It!, substantially below levels demonstrated by recent research models. The proposed system applies transfer learning with EfficientNet-B4, achieving 91.4% top-1 accuracy on Food-101 in practice (Chapter 6).

Gap 4: Absence of Personalised Dietary Recommendations. None of the reviewed deployed systems generates personalised dietary recommendations derived from ML analysis of the individual user's accumulated meal history. The proposed system addresses this through a content-based filtering recommendation module.

Gap 5: Inconsistent Nutritional Data Quality. Several reviewed applications rely in part on user-submitted nutritional entries, which are known to contain inaccuracies. The proposed system sources all nutritional data exclusively from the USDA FoodData Central database.

3.4 Summary

This chapter reviewed five related dietary monitoring systems and evaluated their features, technical approaches, and limitations relative to the objectives of the proposed AI Food Recognition System. The comparative analysis confirms that no existing deployed system simultaneously provides automated portion size estimation, multi-component dish recognition, machine learning-based personalised recommendations, and laboratory-verified nutritional data integration.

The technology review examined candidate classification architectures, training datasets, nutritional data sources, deep learning frameworks, and application platforms. EfficientNet-B4 fine-tuned on Food-101 was selected as the classification backbone; the USDA FoodData Central API was selected as the sole nutritional data source; and PyTorch was selected as the deep learning framework.

Five knowledge gaps were identified: the absence of automated portion estimation, inadequate multi-component dish handling, below-benchmark classification accuracy in deployed applications, the absence of personalised recommendations, and inconsistent nutritional data quality. These gaps directly motivated the technical objectives of the proposed system and the requirements specification presented in Chapter 4.

4 Requirements Specification

4.1 Stakeholders Analysis

A stakeholder is any individual, group, or entity that is affected by the proposed system or that influences its requirements. Table 4.1 identifies the primary stakeholders of the AI Food Recognition System.

Table 4.1: Stakeholder Analysis

Stakeholder	Description	Interaction with System	Importance
End User (General Public)	Individuals seeking to monitor dietary intake for personal health, weight management, or fitness goals. No technical background assumed.	Registers an account, submits food photographs, views nutritional breakdowns, logs meals, sets health goals, and receives personalised recommendations.	Primary user; highest priority.
Healthcare Professional	Dietitians, nutritionists, and general practitioners who may recommend the system to patients between clinical consultations.	May review patient-generated meal history summaries. Does not directly operate the system.	Indirect stakeholder; influences accuracy and data reliability requirements.
System Administrator	The technical individual responsible for deploying, maintaining, and monitoring server-side components.	Manages deployment, monitors performance, updates ML model weights, maintains the USDA API integration.	Essential for system availability and reliability.
Project Supervisor	Maria Yousef, MEU Faculty of Information Technology.	Reviews deliverables, approves scope changes, evaluates the final submitted report and system.	Institutional authority over project direction and academic standards.
MEU Graduation Committee	The academic committee responsible for evaluating graduation projects at MEU.	Assesses the submitted report and final presentation.	Determines project approval and final grade.

4.2 Platform Requirements

The system consists of two subsystems: a client-side native mobile application built with React Native/Expo, and a server-side backend responsible for model inference, nutritional data retrieval, and data storage.

4.2.1 Client-Side Requirements

Table 4.2: Client-Side Platform Requirements

Requirement	Description	Priority
Mobile Operating System	Android 8.0 (API 26) or higher, or iOS 13.0 or higher; distributed as an Expo/React Native package.	Essential
Internet Connection	A stable internet connection is required for API calls to the backend and the USDA API. Minimum 5 Mbps recommended.	Essential
Camera Hardware	A rear camera capable of capturing photographs at a minimum resolution of 640×480 px, with gallery selection support (JPEG/PNG).	Essential
Screen Size	A minimum screen size of 4.7 inches at a minimum resolution of 320×568 px for correct layout rendering.	Essential

4.2.2 Server-Side Requirements

Table 4.3: Server-Side Platform Requirements

Requirement	Description	Priority
Operating System	Ubuntu 22.04 LTS (or equivalent Unix-based environment) for production; Windows/macOS supported for local development.	Essential
Python Runtime	Python 3.11 or 3.12. Required for all backend application code and ML model inference scripts.	Essential
Compute	CPU inference is supported by default; an NVIDIA GPU with CUDA support reduces inference latency where available.	Recommended
Deep Learning Framework	PyTorch 2.x with torchvision for model loading and inference.	Essential
Web Framework	FastAPI for serving REST API endpoints consumed by the client-side application.	Essential

Continued on next page

Table 4.3 – continued from previous page

Requirement	Description	Priority
Database	PostgreSQL 15+ for persistent storage of user accounts, meal logs, and recommendation history.	Essential
USDA API Access	Active internet access to the USDA FoodData Central REST API (api.nal.usda.gov). A valid free API key is required.	Essential

4.3 Functional Requirements

Table 4.4 lists the functional requirements of the system. Each requirement is assigned a unique identifier, specifies the input and output, describes the core process/constraints, and is classified as Essential or Recommended.

Table 4.4: Functional Requirements

ID	Requirement	Input	Output / Process / Constraints	Priority
FR01	User Registration	Email, username, password, health goal	A new account is created; email/username must be unique; password minimum 8 characters.	Essential
FR02	User Authentication	Registered email and password	A JWT session token is issued; accounts locked after 5 consecutive failed attempts.	Essential
FR03	Food Image Submission	JPEG/PNG from camera or gallery	The image is validated and resized to the model's required input dimensions. Max file size 10 MB, min resolution 640×480 px.	Essential
FR04	Food Item Recognition	Preprocessed food image	Returns the top predicted food item(s) with confidence scores; if confidence is below threshold (60%), up to three alternatives are presented.	Essential
FR05	Portion Size Estimation	Predicted food label and image	Returns a calibrated mass range estimate (min/max grams) for the identified food item.	Essential

Continued on next page

Table 4.4 – continued from previous page

ID	Requirement	Input	Output / Process / Constraints	Priority
FR06	User Portion Confirmation	Estimated portion range	The user accepts or adjusts the portion using an on-screen stepper; the confirmed portion is stored.	Essential
FR07	Cooking Method Selection	List of predefined cooking methods	The user selects the cooking method (grilled, fried, boiled, etc.); the selection is appended to the USDA query.	Essential
FR08	Nutritional Data Retrieval	Confirmed food item and cooking method	Queries the USDA FoodData Central API and returns calories, protein, carbohydrate, and fat scaled to the confirmed portion.	Essential
FR09	Nutritional Summary Display	Retrieved macronutrient values	Renders a calorie ring and macronutrient bars with a visible disclaimer.	Essential
FR10	Meal Log Persistence	Confirmed meal items and macronutrients	The meal record is stored; a minimum of 7 confirmed logs is required before recommendations are generated.	Essential
FR11	Meal History View	Authenticated user session	Retrieves and displays a paginated, chronological list of logged meals with per-meal calorie summaries and delete support.	Essential
FR12	Personalised Recommendation	Accumulated meal history (≥ 7 logs)	Generates up to five personalised dietary insights aligned with the user's health goal; all include the statutory disclaimer.	Essential
FR13	Health Goal Management	Selected health goal	The user's health goal is stored and used as the recommendation target; may be updated at any time.	Essential
FR14	Account Data Deletion	Authenticated user request	All personal data (account, meal logs, recommendations) is permanently and irreversibly deleted.	Essential

4.4 Non-Functional Requirements

Performance. The system returns a food recognition result and nutritional estimate within a few seconds of image submission under normal load, dependent on device com-

pute (CPU vs GPU inference).

Accuracy. The food classification module targets a minimum top-1 accuracy of 80% on the Food-101 test partition; the trained model achieved 91.4% (Chapter 6). Nutritional values are sourced exclusively from the laboratory-verified USDA FoodData Central database.

Usability. The interface allows a first-time user with no prior experience to complete the full food-logging workflow — from image submission to meal record confirmation — within no more than three user interactions.

Security. All user passwords are stored as salted bcrypt hashes. All client–server communication uses HTTPS/TLS in production. Meal history is accessible only to the authenticated account holder via JWT-scoped queries.

Scalability. The server-side architecture is designed to support a target concurrent user load of at least 50 simultaneous active sessions without significant degradation of response time.

Reliability and Availability. A local PostgreSQL cache of frequently queried USDA entries ensures continued nutritional data retrieval during periods of temporary API unavailability.

Portability. The React Native/Expo mobile application runs on both Android and iOS from a single shared TypeScript codebase without platform-specific modifications.

Maintainability. Backend source code follows PEP 8 style guidelines with inline documentation. ML model weights are stored as versioned artefacts to allow rollback to a previous version if needed.

4.5 Other Requirements

API Usage Constraints. The system interacts with the USDA FoodData Central REST API using the standard JSON response format, with a valid API key passed as a query parameter, and does not exceed the USDA rate limit of 3,600 requests per hour per key.

Data Storage Format. All user meal records are stored in a relational database in third normal form. Food item names, macronutrient values, and portion estimates are stored as structured fields to facilitate aggregation and recommendation computation. Images are not stored permanently; image data is processed in memory and discarded after a meal record is confirmed.

Privacy and Data Protection. The system collects only the minimum personal data required for operation: username, email, health goal, and meal log records. No biometric data, precise location data, or payment information is collected. All stored personal data is deletable upon user request.

Disclaimer Requirement. The system displays a clearly visible disclaimer on the Nutrition and Insights screens stating that all nutritional information and dietary suggestions are for informational purposes only and do not constitute medical or clinical nutritional advice.

5 Architecture and Design

This chapter presents the complete architecture and design of the AI Food Recognition System: its structural decomposition into components, the data model underpinning persistent storage, the workflows governing system behaviour, the interface designs presented to end users, the technology stack, and the development and deployment environment. All design decisions are grounded in the requirements defined in Chapter 4 and the technology review conducted in Chapter 3.

5.1 System Architecture

The AI Food Recognition System is structured as a three-tier mobile application comprising a client presentation layer, a backend processing layer, and a data layer. This layered decomposition separates user-facing concerns from business logic and data persistence, promoting modularity, testability, and independent scalability of each tier.

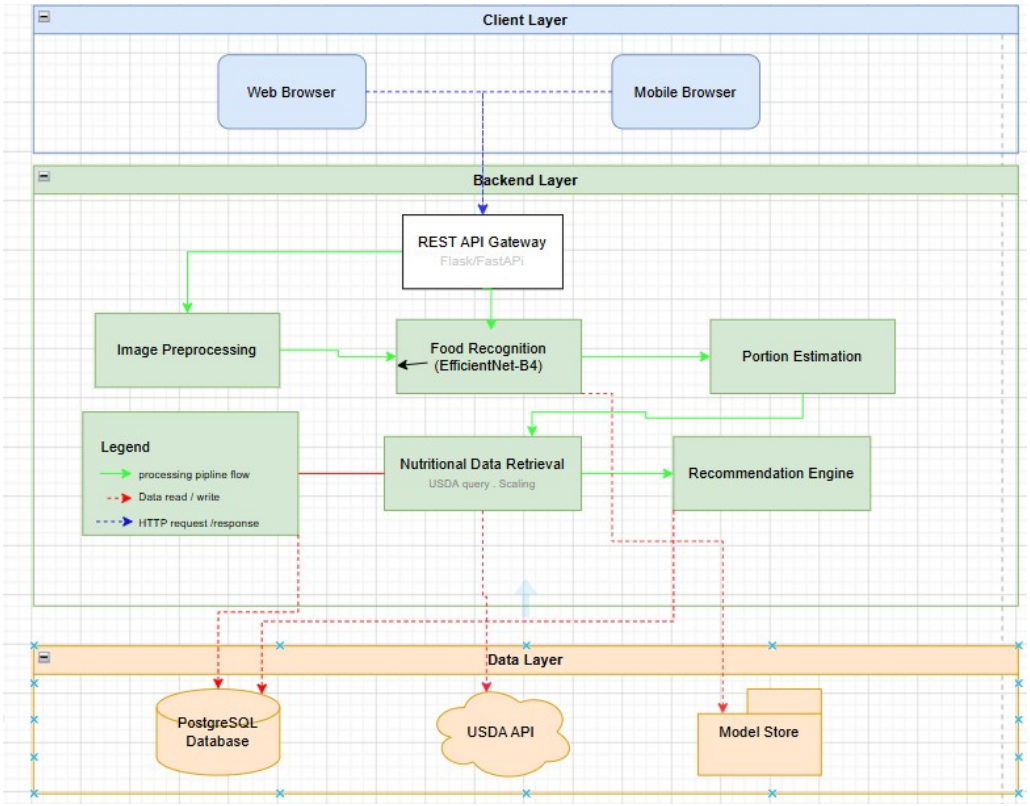


Figure 5.1: High-Level System Architecture Diagram

The client layer contains a native mobile application built with React Native (Expo), deployable on Android and iOS as an installable package. The backend processing layer hosts four specialised processing modules — image preprocessing, food recognition, portion estimation, and the recommendation engine — exposed through a unified REST

API gateway. The data layer encompasses a PostgreSQL relational database, the USDA FoodData Central API, and a versioned model repository for trained weight artefacts.

All inter-module communication within the backend is synchronous function call invocation. Communication between the client and the backend is conducted exclusively over HTTPS using RESTful JSON APIs. The USDA FoodData Central API is accessed via outbound HTTP requests, with a local PostgreSQL cache providing fallback coverage during periods of API unavailability.

5.2 Data Design

The data model is designed in third normal form (3NF) to eliminate insertion, update, and deletion anomalies. It comprises five core tables: user, meal_log, meal_item, food_item, and recommendation.

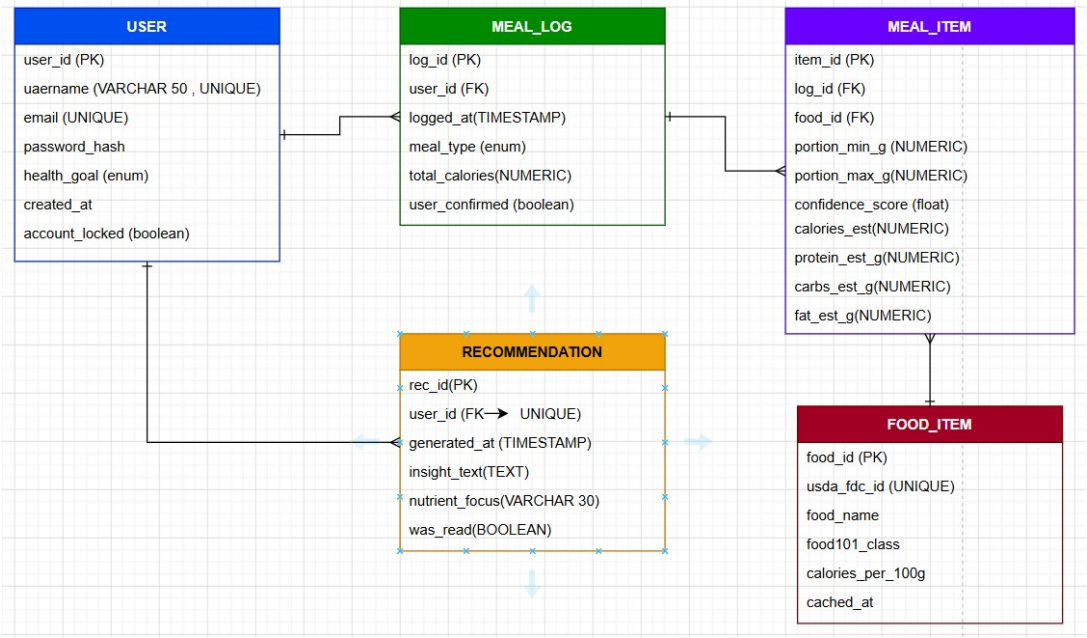


Figure 5.2: Entity-Relationship Diagram — AI Food Recognition System

The user table stores account credentials, health goals, and account status. Each user may have many meal_log records (one-to-many). Each meal_log record represents a single logged meal and stores aggregated macronutrient totals computed from its constituent items. Each meal_log contains one or more meal_item records, where each item references a food_item entry and stores per-item portion estimates and derived macronutrient values. The food_item table serves as a local USDA nutritional cache, mapping Food-101 class labels to USDA identifiers. The recommendation table stores generated dietary insights linked to the target user.

Images submitted by users are explicitly excluded from persistent storage. Image data is held in server memory only for the duration of the recognition and estimation pipeline and is discarded immediately after a meal record is confirmed, minimising storage requirements and privacy risk.

5.2.1 Database Schema (3NF)

Table 5.1 presents the full database schema. Foreign key relationships are enforced at the database level with ON DELETE CASCADE semantics where the parent record owns the child records (user → meal_log → meal_item), and ON DELETE RESTRICT for shared reference records (food_item).

Table 5.1: Database Schema — Third Normal Form

Table	Column	Type	Constraint	Description
user	user_id	SERIAL	PK	Auto-incrementing primary key.
user	username	VARCHAR(50)	UNIQUE, NOT NULL	Unique display name chosen at registration.
user	email	VARCHAR(100)	UNIQUE, NOT NULL	Email address; used for authentication.
user	password_hash	VARCHAR(255)	NOT NULL	Salted bcrypt hash of the user password.
user	health_goal	ENUM	NOT NULL	weight_loss — maintenance — weight_gain.
user	created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp.
user	account_locked	BOOLEAN	DEFAULT FALSE	Set TRUE after 5 consecutive failed logins.
meal_log	log_id	SERIAL	PK	Auto-incrementing primary key.
meal_log	user_id	INTEGER	FK → user	Owner of this meal record.
meal_log	logged_at	TIMESTAMP	NOT NULL	Time the meal was submitted.
meal_log	meal_type	ENUM	NOT NULL	breakfast — lunch — dinner — snack.
meal_log	total_calories	NUMERIC(7,2)	—	Aggregated calorie estimate for the meal.

Continued on next page

Table 5.1 – continued from previous page

Table	Column	Type	Constraint	Description
meal_log	user_confirmed	BOOLEAN	DEFAULT FALSE	TRUE once user has confirmed the recognition result.
meal_item	item_id	SERIAL	PK	Auto-incrementing primary key.
meal_item	log_id	INTEGER	FK → meal_log	Parent meal log record.
meal_item	food_id	INTEGER	FK → food_item	Identified food item.
meal_item	portion_min_g	NUMERIC(6,1)	—	Lower bound of the portion size estimate.
meal_item	portion_max_g	NUMERIC(6,1)	—	Upper bound of the portion size estimate.
meal_item	portion_confirmed_g	NUMERIC(6,1)	—	User-confirmed portion mass.
meal_item	confidence_score	FLOAT	—	Classifier confidence for this food class (0–1).
food_item	food_id	SERIAL	PK	Auto-incrementing primary key.
food_item	usda_fdc_id	VARCHAR(20)	UNIQUE	USDA FoodData Central identifier.
food_item	food_name	VARCHAR(200)	NOT NULL	Human-readable food name from USDA.
food_item	food101_class	VARCHAR(50)	—	Corresponding Food-101 class label.
food_item	calories_per_100g	NUMERIC(7,2)	—	USDA lab-verified calorie value per 100 g.
food_item	cached_at	TIMESTAMP	—	When the USDA record was cached locally.

Continued on next page

Table 5.1 – continued from previous page

Table	Column	Type	Constraint	Description
recommendation	rec_id	SERIAL	PK	Auto-incrementing primary key.
recommendation	user_id	INTEGER	FK → user, UNIQUE	Target user for this recommendation.
recommendation	generated_at	TIMESTAMP	NOT NULL	When the recommendation was generated.
recommendation	insight_text	TEXT	NOT NULL	Human-readable dietary insight.
recommendation	nutrient_focus	VARCHAR(30)	—	Nutrient dimension targeted (e.g., protein, fibre).
recommendation	was_read	BOOLEAN	—	Whether the user has viewed this insight.

5.3 Workflow and Process Design

This section presents the object-oriented design of the system using four UML diagram types: Use Case, Sequence, Activity, and Component.

5.3.1 Use Case Diagram

Figure 5.3 illustrates the use cases of the system. The primary actor is the End User, who registers, logs in, submits food photographs, confirms recognition and portion results, selects the cooking method, logs meals, views history, updates the health goal, and receives recommendations. ⟨include⟩ relationships indicate mandatory sub-behaviours: submitting a food image always includes running ML inference, and logging a meal always includes retrieving nutritional data.

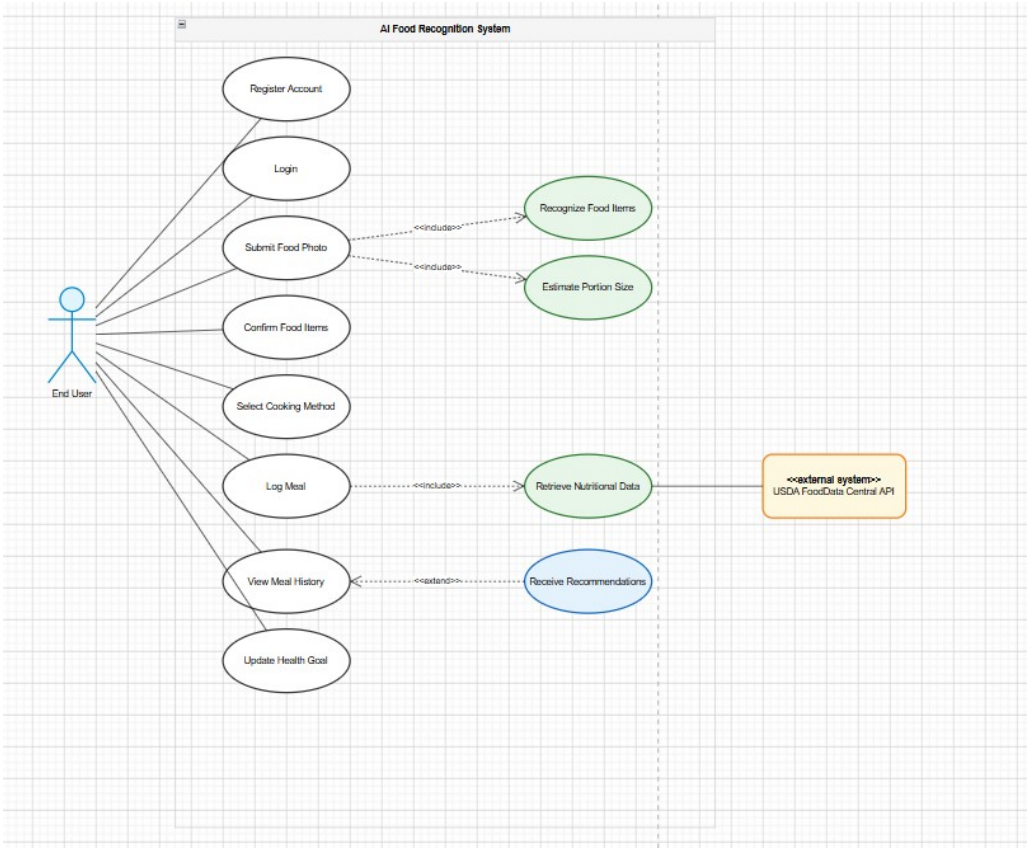


Figure 5.3: Use Case Diagram — AI Food Recognition System

5.3.2 Sequence Diagram

Figure 5.4 presents the sequence diagram for the primary food recognition and nutritional estimation flow. A user submits a food photograph via the React Native application, which sends an HTTPS POST request to the REST API Gateway. The gateway validates the session token and file constraints before forwarding the image to the classification module. The EfficientNet-B4 model performs inference and returns the top predicted food labels with confidence scores, which are passed to the portion estimation module. The portion module then queries the USDA FoodData Central API (or local cache) for macronutrient values. The assembled result is returned through the API Gateway to the client as structured JSON.

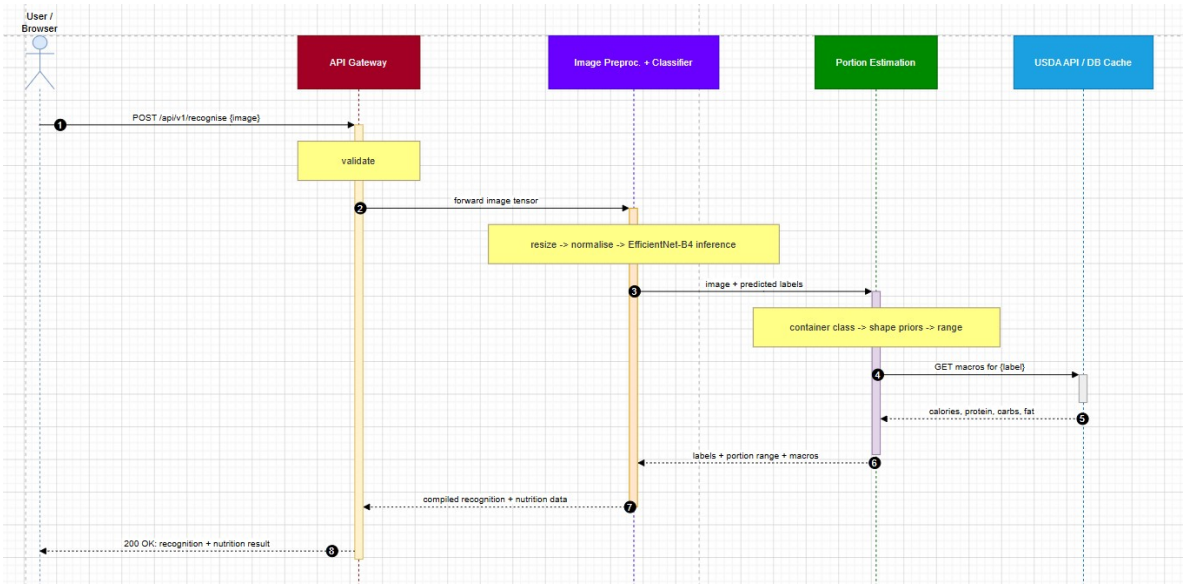


Figure 5.4: Sequence Diagram — Food Recognition and Nutritional Estimation Flow

5.3.3 Activity Diagram

Figure 5.5 presents the activity diagram for the end-to-end user workflow. The workflow begins when the user opens the application and authenticates, captures or uploads a food photograph, and the system runs the EfficientNet-B4 inference pipeline. A decision node branches on whether the top-1 confidence meets the defined threshold: if it does, the result is presented directly; if not, up to three alternative predictions are displayed. The workflow proceeds through portion estimation, user confirmation, USDA macronutrient retrieval, nutritional summary display, meal log persistence, and a conditional check on whether the user has reached the minimum seven logged meals required for recommendations.

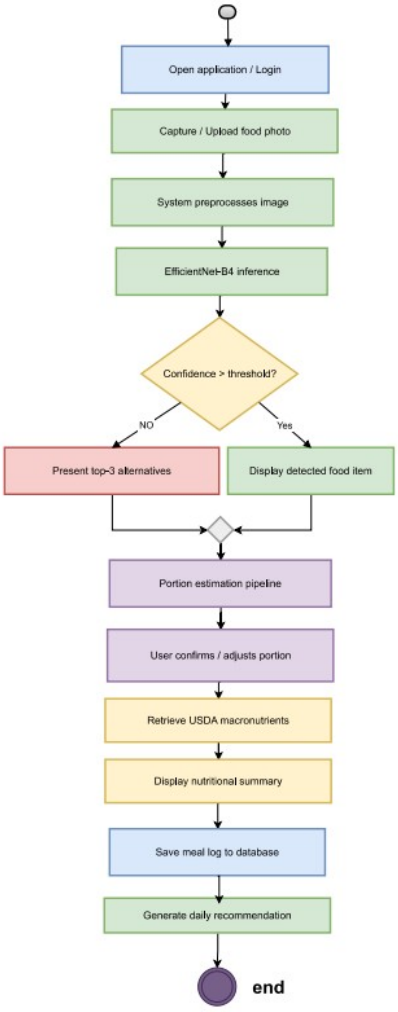


Figure 5.5: Activity Diagram — End-to-End User Workflow

5.3.4 Component Diagram

Figure 5.6 illustrates the major software components and their dependencies. The Frontend UI component is a React Native project containing screen components for image capture, result rendering, and the meal history dashboard. The API Gateway component contains the authentication middleware, route handlers, and rate limiter. The ML Pipeline component encapsulates the Python processing modules. The Data Services component abstracts all external data access behind the USDA API client and the database ORM.

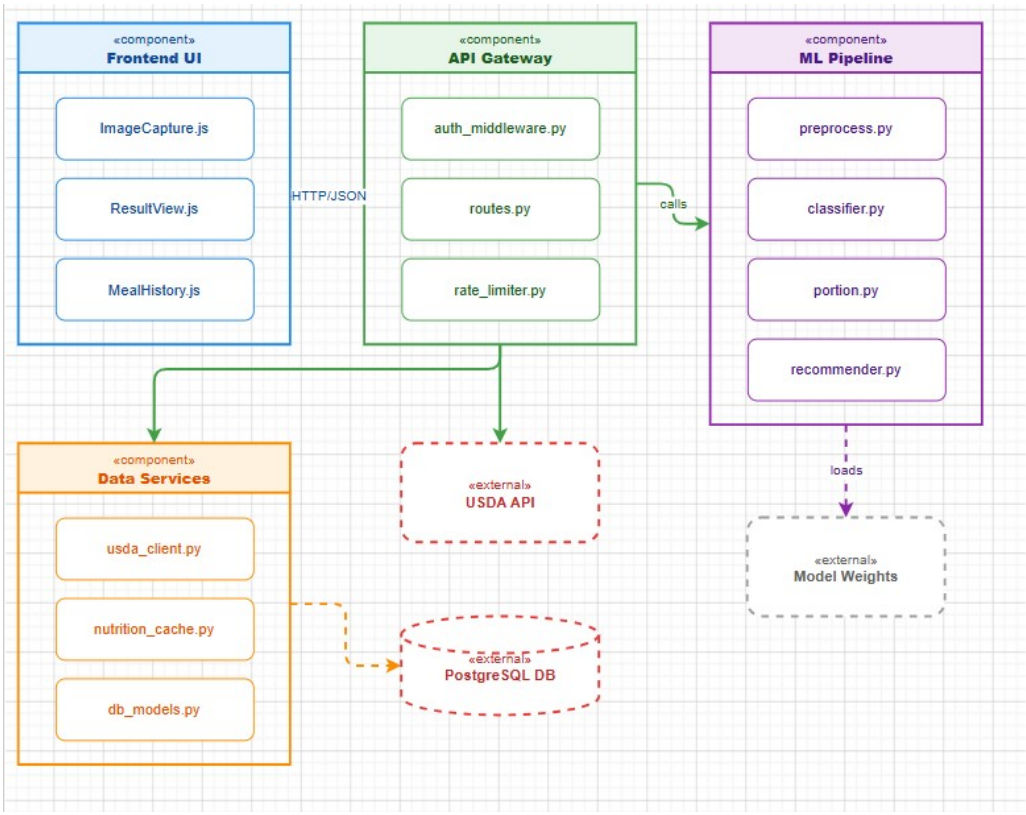


Figure 5.6: Component Diagram — AI Food Recognition System

5.3.5 Class and Module Overview

Table 5.2 presents the module-level overview, specifying the package membership, design pattern applied, and primary responsibility for each backend module.

Table 5.2: Module / Class Overview — Backend Processing Layer

Module / File	Package	Pattern	Responsibility
preprocess.py	ml_pipeline	Pipeline	Resize, normalise, and augment input images to 380×380 tensors compatible with EfficientNet-B4.
classifier.py	ml_pipeline	Strategy	Load fine-tuned EfficientNet-B4 weights; perform food classification across the 101 Food-101 categories; return top-k predictions with confidence scores.
portion.py	ml_pipeline	Chain of Responsibility	Execute container classification → reference-object calibration → shape-prior estimation pipeline; return a portion range in grams.

Continued on next page

Table 5.2 – continued from previous page

Module / File	Package	Pattern	Responsibility
recommender.py	ml_pipeline	Observer	Aggregate confirmed meal records into a daily macro-average profile; compare against the target distribution for the user's health goal; return up to five dietary insights, subject to the 7-meal minimum (FR10).
food_gate.py	ml_pipeline	Guard / Gate	Pre-classification check that verifies the submitted image plausibly contains food before running the main classifier, reducing spurious high-confidence misclassifications on non-food images.
usda_client.py	data_ services	Adapter	Abstract USDA FoodData Central REST API calls; map food labels to USDA entries; return structured macronutrient dictionaries.
nutrition_ cache.py	data_ services	Cache	Store frequently queried USDA entries in PostgreSQL; serve cached values during API unavailability.
db_models.py	data_ services	ORM (SQLAlchemy)	Define SQLAlchemy ORM classes for User, MealLog, MealItem, FoodItem, and Recommendation in 3NF.
auth_ middleware.py	api_ gateway	Middleware	Validate JWT session tokens on protected endpoints; enforce account lock-out after five consecutive failed login attempts.
routes.py	api_ gateway	Controller	Define FastAPI route handlers for /recognise, /nutrition, /meals, /recommendations, and /auth endpoints.
rate_limiter.py	api_ gateway	Decorator	Enforce the USDA API rate limit (3,600 requests/hour/key) via request counting.

5.3.6 Recommendation Engine Design

The recommendation engine, implemented in recommender.py, operates against the user's accumulated meal history stored in PostgreSQL, in three stages. First, it aggregates confirmed meal records over a rolling window into a daily macro-average profile vector (calories, protein, carbohydrate, fat), performed only once the minimum threshold of seven logged meals is reached. Second, it computes the signed difference between the user's profile and the target macro distribution associated with the stated health goal, identifying the dimension with the largest deviation as the primary nutritional gap. Third, it queries the food_item table to identify food items that would reduce the identified gap and generates plain-language insight strings, all including the statutory disclaimer.

5.4 Interface Design

The user interface is a native mobile application built with React Native. All screens are optimised for smartphone form factors and follow React Native's cross-platform layout system for consistent rendering on Android and iOS. Navigation between screens uses the React Navigation library, with a bottom tab bar (Home, History, Capture, Insights, Profile) and a nested capture stack (Camera → Recognition Results → Nutrition Summary).

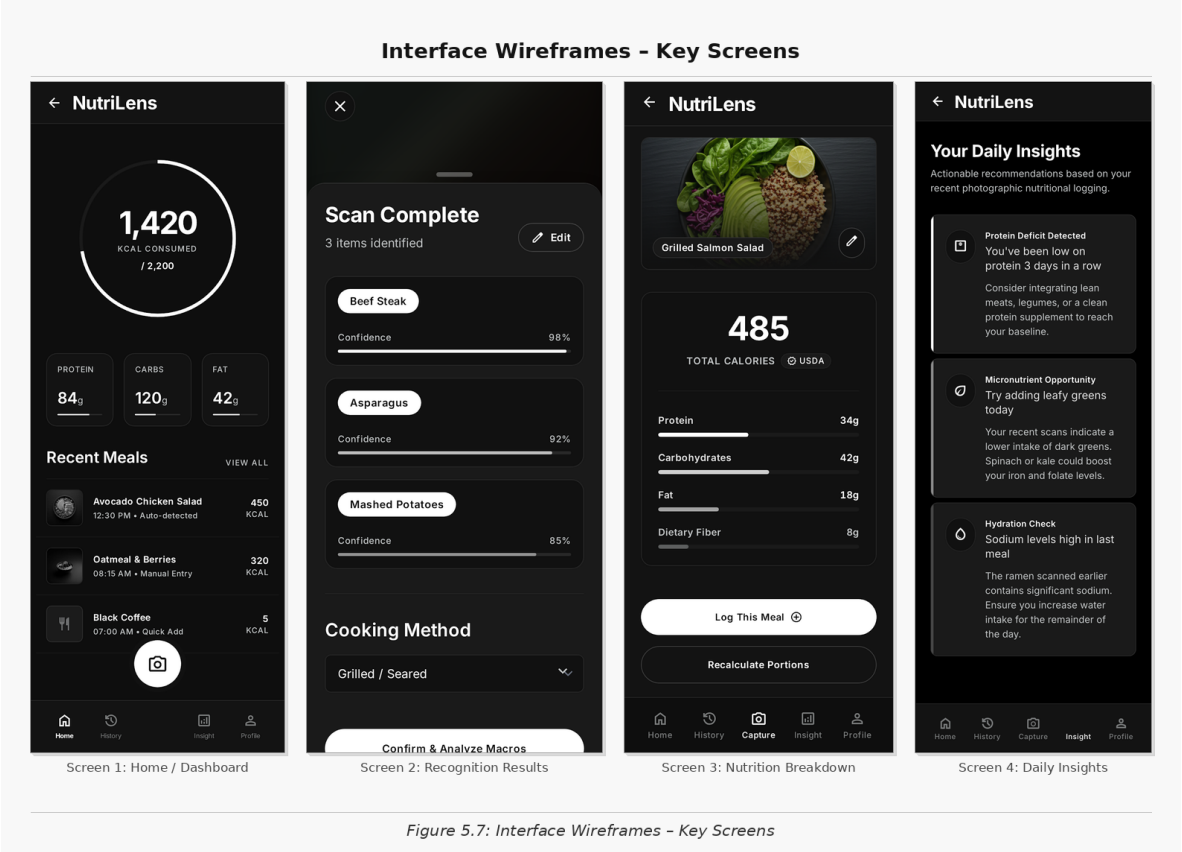


Figure 5.7: Interface Wireframes — Key Screens

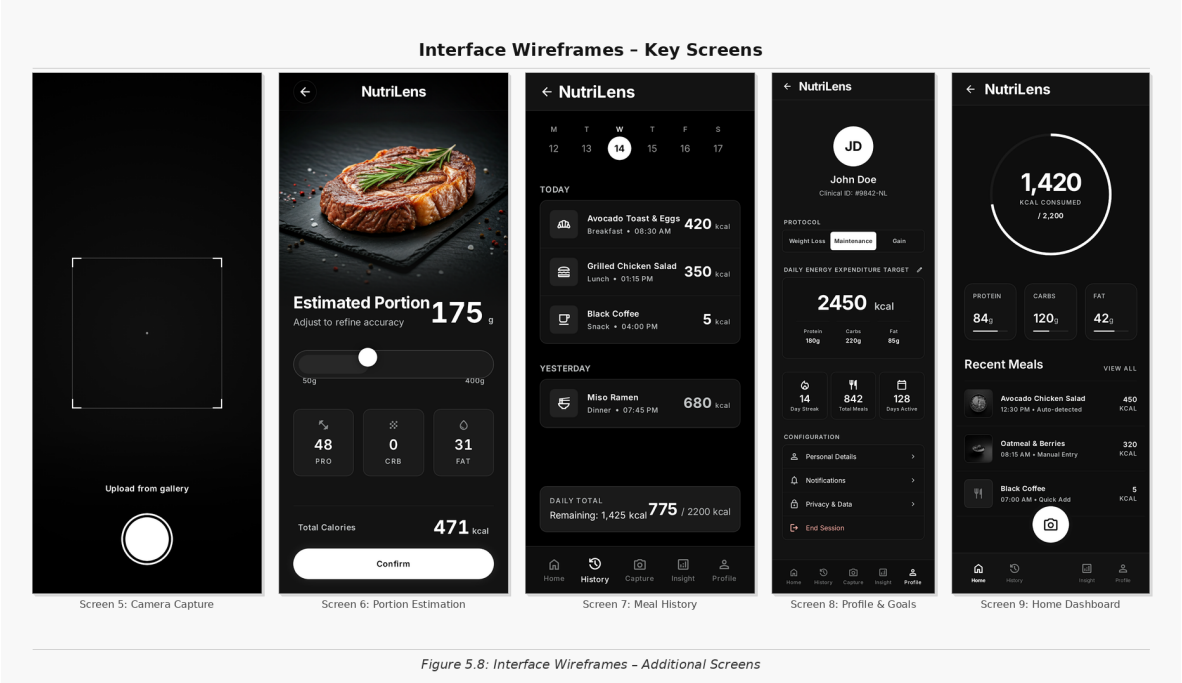


Figure 5.8: Interface Wireframes — Additional Screens

The navigation flow follows the sequence: Home → Camera Capture → Recognition Results → Nutrition Summary → (back to Home). Users may access Meal History, Insights, and Profile at any time from the persistent bottom navigation bar. The Nutrition and Insights screens render the disclaimer text directly from the API response rather than from a local copy, so the two cannot drift apart.

5.5 Technology Stack

Table 5.3 presents the complete technology stack, with justification for each selection.

Table 5.3: Technology Stack Justification

Component	Selected	Alternative(s)	Justification
Classification Model	EfficientNet-B4 (PyTorch)	ResNet-50, VGGNet-16, ViT-Base	Favourable balance of accuracy (83.0% ImageNet top-1) and parameter efficiency (19M params); ViT excluded due to data-hungry pre-training.
Training Dataset	Food-101	ISIA Food-500, UEC Food-100	Standard benchmark (101,000 images, 101 classes) enabling comparison with published results; feasible on the available GPU budget.

Continued on next page

Table 5.3 – continued from previous page

Component	**Selected**	**Alternative(s)**	**Justification**
Nutritional Database	USDA FoodData Central API	Nutritionix API, Open Food Facts	Lab-verified macronutrient values, free unlimited public API, separate entries per cooking method.
Deep Learning Framework	PyTorch 2.x	TensorFlow / Keras	Flexible dynamic computation graph, strong EfficientNet fine-tuning community support, first-class Colab/Kaggle support.
Mobile Frontend	React Native (Expo SDK 54)	Flutter, Ionic	Single TypeScript codebase deployable on Android and iOS; native camera/gallery APIs; React Navigation for screen management.
Backend API Framework	FastAPI	Flask, Django, Node.js/Express	Lightweight Python-native REST framework consistent with the ML stack; automatic OpenAPI documentation.
Database	PostgreSQL 15+	MySQL, MongoDB	ACID-compliant relational database suits the structured meal-log/user-profile schema defined in 3NF.
Compute Environment	Google Colab Pro + Kaggle	Local GPU, AWS EC2	Cost-effective GPU access (approx. $40 total) accessible via browser without local hardware requirements.
Version Control	GitHub (Git)	—	Industry-standard VCS; three-branch strategy (main / develop / feature/*).
Model Repository	GitHub Releases	DVC, MLflow	Model weight files (.pth) stored as versioned release assets, keeping large binaries out of the code commit history.

The selected stack is entirely open-source, with the sole recurring cost being the Google Colab Pro subscription for GPU-accelerated model training (approx. $40 total), consistent with the cost estimation in Section 2.5. No proprietary dependencies are introduced.

5.6 Development and Deployment Environment

5.6.1 Development Environment

Model training and experimentation were conducted in Google Colab Pro and Kaggle Notebook environments with GPU acceleration. The local development environment for backend code uses a Python 3.11/3.12 virtual environment, with VS Code as the primary editor. The React Native/Expo mobile application was developed using Node.js and the Expo CLI, and tested on a physical Android device via Expo Go.

5.6.2 Version Control Strategy

The GitHub repository implements a three-branch strategy: main (protected, stable, supervisor-reviewed), develop (active integration branch), and feature/* (short-lived branches for individual tasks, e.g. feature/m1-m2-db-auth). Model weight files (.pth) are kept out of the main code repository (excluded via .gitignore) to prevent large binaries from inflating the commit history; team members exchange them directly and place them in backend/models/ locally. Commit messages follow the Conventional Commits specification (feat:, fix:, docs:, test:).

5.6.3 Deployment Target

During development and evaluation, the backend runs locally via Uvicorn against a Dockerised PostgreSQL instance, with the mobile application connecting over the local network (LAN) using the development machine's IP address. This mirrors the intended production deployment shape — a containerised FastAPI/PostgreSQL backend behind HTTPS — while keeping the evaluation environment simple and cost-free. The mobile application is run through Expo Go for testing and can be built as a signed Android APK for wider distribution.

6 Implementation

6.1 General Implementation Description

This chapter documents the implementation phase of the AI Food Recognition System (NutriLens), building directly on the architecture and design specified in Chapter 5. Where Chapter 5 described what the system was designed to do, this chapter describes what was actually built, the tools used to build it, and how the working prototype behaves end to end on a physical Android device.

The implemented system consists of two cooperating parts: a native mobile application and a backend service. The mobile application is built with React Native on the Expo SDK, giving a single TypeScript codebase that runs on Android and iOS without platform-specific rewrites. It communicates over HTTPS/JSON with a FastAPI backend, which in turn talks to a PostgreSQL database for persistent storage and to the USDA FoodData Central API for verified nutrition data. The food recognition capability itself is powered by a fine-tuned EfficientNet-B4 convolutional neural network, trained on the Food-101 dataset using transfer learning from ImageNet-pretrained weights.

Development responsibilities were divided between the two team members along the same lines set out in the project plan (Chapter 2): the backend, database schema, authentication, and the machine learning pipeline (classification, portion estimation, and the food-gate pre-check) were implemented by Mohannad Alhemsi, while the mobile application — its eleven screens and the shared components they are built from, their navigation, and their integration against the live backend — was implemented by Shatha Alghrair. Both team members collaborated on end-to-end testing, defect resolution, and the environment setup that lets the two halves talk to each other over the local network during development.

6.1.1 Tools and Technologies Used

Table 6.1 summarises every tool, framework, and library used in the implementation, and the role each one plays in the finished system.

Table 6.1: Implementation Tools and Technologies

Tool / Technology	**Specifics**	**Role in the Implementation**
Mobile Frontend	React Native (Expo SDK 54), TypeScript	Cross-platform mobile app (Android/iOS) from a single codebase; native camera and gallery access.
Navigation	React Navigation (bottom tabs + nested native stack)	Home / History / Capture / Insights / Profile tabs; Camera → Recognition → Nutrition Summary flow.

Table 6.1: Implementation Tools and Technologies

Tool / Technology	**Specifics**	**Role in the Implementation**
State & Data Fetching	TanStack React Query	Server-state caching, background refetch, and mutation handling (login, meal logging, profile updates).
Backend API	FastAPI (Python)	REST API gateway: authentication, recognition, nutrition, meals, recommendations, and user endpoints.
Database	PostgreSQL 15 (Dockerised) + SQLAlchemy ORM + Alembic	Persistent storage for users, meal logs, meal items, food items, and recommendations, in 3NF.
Authentication	JWT (JSON Web Tokens) + bcrypt	Stateless session tokens; salted password hashing; account lock-out after 5 failed login attempts.
Deep Learning Framework	PyTorch + torchvision	Model definition, fine-tuning, and inference for the food classification model.
Classification Model	EfficientNet-B4 (transfer learning on Food-101)	Identifies the food item in a submitted photograph across 101 categories.
Nutrition Data Source	USDA FoodData Central REST API	Laboratory-verified calories, protein, carbohydrate, and fat values per confirmed food item and cooking method.
Image Handling	expo-camera, expo-image-picker, expo-image-manipulator	Live camera capture, gallery selection, and client-side image resizing/compression before upload.
Version Control	Git / GitHub (main, develop, feature/* branches)	Collaborative development between the two team members with a linear commit history.
Containerisation	Docker / Docker Compose	Runs the PostgreSQL database in an isolated, reproducible local environment.
Computer Vision	OpenCV, NumPy, Pillow	Container rim detection, the classical segmentation fallback, mask area measurement, and image decoding and validation on upload.

Table 6.1: Implementation Tools and Technologies

Tool / Technology	**Specifics**	**Role in the Implementation**
Pretrained Vision Models	torchvision (MobileNetV3, DeepLabV3, Faster R-CNN)	Off-the-shelf food-gate and reference-object detection; backbones for the segmenter and the container classifier.
Training Data Collection	FiftyOne / Open Images V7; HuggingFace Datasets / FoodSeg103	Automated harvesting of container crops and food segmentation masks, avoiding manual labelling.
API Schemas & Validation	Pydantic	Declarative request and response contracts, so the wire format cannot drift away from the code that serves it.
ASGI Server	Uvicorn	Serves the FastAPI application during development and testing.
HTTP Client	httpx	Outbound calls to the USDA FoodData Central API, with timeout and retry handling.
Quality Tooling	pytest, ruff, black, mypy	Automated test suite, linting, formatting, and static type checking (Chapter 7).

6.1.2 The Machine Learning Models

The core of the recognition module is an EfficientNet-B4 convolutional neural network, fine-tuned on the Food-101 dataset (101 food categories, 101,000 images). EfficientNet-B4 was chosen over the alternatives evaluated in Chapter 3 (ResNet-50, VGGNet-16, ViT-Base) for its favourable balance of accuracy and parameter count, which keeps inference practical on CPU-only hardware during development and testing.

The trained model achieved a top-1 accuracy of 91.4% on the Food-101 test partition — comfortably above the 80% target set in the non-functional requirements (Section 4.4) and well above the 46% top-1 accuracy reported for comparable commercial applications (FoodVisor, Lose It!) in the literature review (Chapter 3). A ResNet-50 model was also trained in parallel as a comparison baseline, reaching approximately 83% validation accuracy; this confirmed that the additional complexity of EfficientNet-B4 translated into a real accuracy gain rather than overfitting.

A confidence threshold of 60% is applied to every prediction. Above the threshold, the model's top-1 prediction is shown directly. Below it, the system falls back to presenting the top three alternative predictions and lets the user pick the correct one — directly implementing FR04 from the requirements specification. A newly added food-gate check runs before classification and verifies that the submitted image plausibly contains food at all, reducing confidently-wrong predictions on non-food photographs.

Models in the served pipeline. The food classifier is the largest of the models the system runs, but not the only one. Table 6.2 lists all five, distinguishing the three trained

for this project from the two used off the shelf.

Table 6.2: Machine Learning Models in the Served Pipeline

Model	Architecture	Role in the System	Training
Food classifier	EfficientNet-B4 (19 M parameters, 380 px input)	Identifies the dish across the 101 Food-101 categories.	Fine-tuned for this project on Food-101; 91.4% top-1 test accuracy.
Food segmenter	DeepLabV3-MobileNetV3-Large (512 px input)	Separates food pixels from plate and background so the food's area can be measured.	Fine-tuned for this project on FoodSeg103, its 104 classes collapsed to a binary food/background target.
Container classifier	MobileNetV3-Small	Identifies the container (plate or bowl) supplying the size prior used for scale.	Fine-tuned for this project on plate and bowl crops harvested from Open Images V7, about 1,500 per class.
Food gate	MobileNetV3-Large	Open-set check that the photograph shows food at all, before classification runs.	Off the shelf; ImageNet-1k weights, no training required.
Reference detector	Faster R-CNN MobileNetV3	Locates a fork, spoon or knife for reference-object scale calibration.	Off the shelf; COCO weights, no training required.

Training procedure. All three trained models follow the same transfer-learning recipe. Training runs in two phases: a three-epoch warm-up in which the pretrained backbone is frozen and only the new classification head is trained at a learning rate of 1×10^{-3}, followed by a full fine-tune of the whole network at 3×10^{-4}, using AdamW with a weight decay of 1×10^{-4} and a learning-rate schedule that warms up linearly over two epochs and then decays on a cosine curve. Regularisation is deliberately heavy, because Food-101's 750 training images per class are readily memorised: random resized cropping (scale 0.65–1.0), horizontal flipping, RandAugment with two operations at magnitude 9, random erasing at probability 0.25, MixUp ($\alpha = 0.2$) and CutMix ($\alpha = 1.0$) trained against soft targets, and label smoothing of 0.1. An exponential moving average of the weights (decay 0.999) is maintained and evaluated alongside the raw weights, mixed-precision arithmetic with gradient accumulation keeps the batch size workable on a single GPU, training stops early after eight epochs without improvement, and the final evaluation applies horizontal-flip test-time augmentation. The evaluation transform used for that measurement — resize to 400, centre-crop to 380, normalise with ImageNet statistics — is precisely the transform the backend applies at serving time (Section 6.3, Step 3), so the accuracy reported in Chapter 7 is the accuracy the deployed system actually delivers.

6.1.3 Environment and Execution

During development and testing, the system runs across three cooperating processes on the same local network: a Dockerised PostgreSQL instance, a FastAPI backend served locally via Uvicorn, and the Expo development server feeding the mobile application on a physical Android device through Expo Go. This mirrors the intended production deployment shape (a containerised FastAPI/PostgreSQL backend behind HTTPS) while keeping the development loop simple and free of hosting costs, consistent with the cost estimate in Section 2.5.

6.2 Pipeline Implementation Description

Figure 6.1 presents the implemented recognition-to-recommendation pipeline as a flowchart, from the moment a user captures or uploads a meal photo through to the meal being logged and, once enough history has accumulated, a personalised insight being generated. The diagram shows the pipeline steps only; a detailed explanation of each step is given separately in Section 6.3.

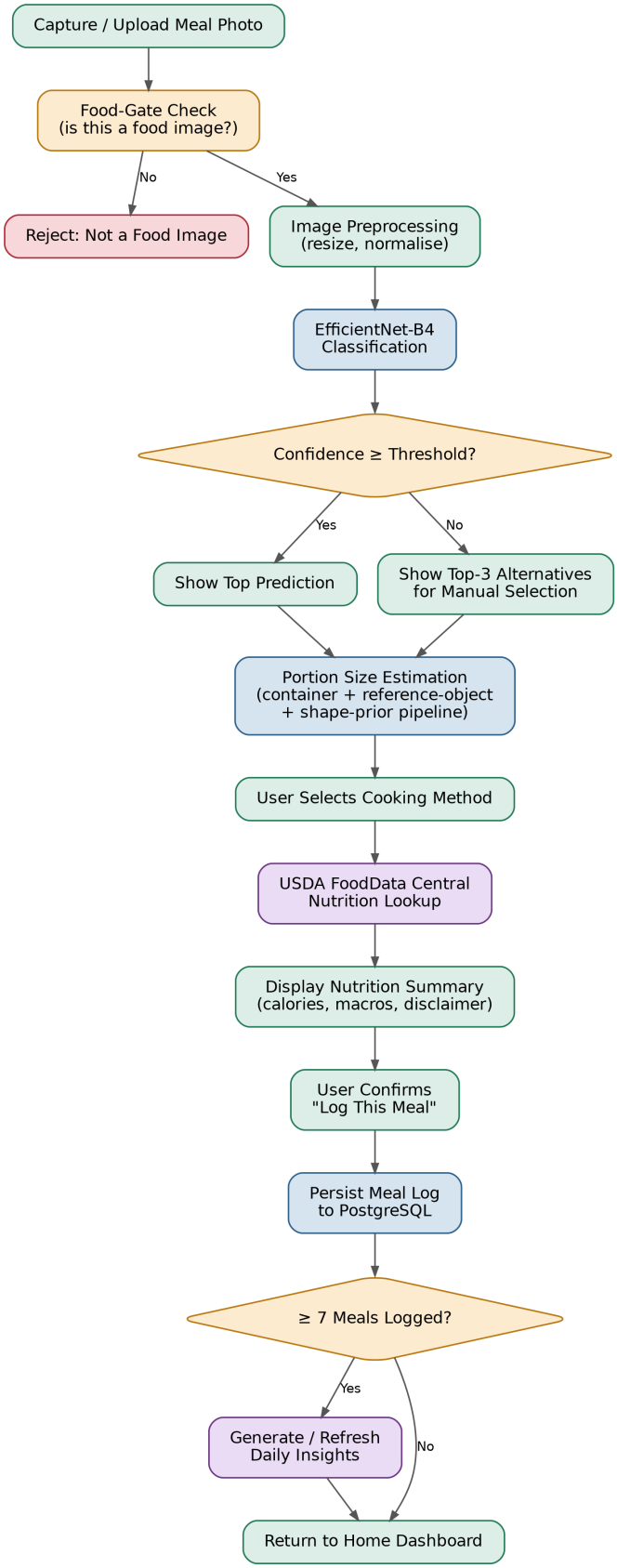


Figure 6.1: End-to-End Recognition and Logging Pipeline

6.3 Model Implementation

This section explains each step of the pipeline shown in Figure 6.1 in detail: what the step does, how it was implemented, and which requirement or design decision from Chapters 4 and 5 it satisfies.

Step 1: Capture / Upload Meal Photo

The mobile application's Camera screen, built with expo-camera, gives the user a live viewfinder with a shutter button and a "Gallery" option for selecting an existing photo (expo-image-picker). This satisfies FR03: the app accepts a JPEG/PNG image either freshly captured or chosen from the device gallery, and immediately checks it against the minimum resolution constraint (640×480 px) client-side before doing anything else, so an unusable photo is rejected before it ever reaches the network.

Step 2: Food-Gate Check

EfficientNet-B4 is a closed-set classifier: its softmax is defined over exactly 101 food categories and must sum to one no matter what the camera is pointed at, so it has no way to express "this is not food". Photographed a laptop, it answers with whichever of the 101 foods it has a word for. Step 2 supplies the missing half — an open-set pre-filter that decides whether the photograph shows food at all, before the specialist model is asked which food it is. This was a defensive addition made during implementation and was not part of the original Chapter 5 design.

The obvious guard — rejecting predictions whose confidence is low — does not work, and the counter-example was measured during development. One photograph of a laptop scored 0.05 on its best guess and would have been caught by any confidence floor. A second, taken from a closer angle, is the informative one: it scored 0.46 on "apple pie" with the runner-up at 0.09 — a high score and a wide margin, on an object that is plainly not food. Confidence answers which of the 101 categories is most likely; it never answers whether any of them applies. No threshold on it can separate the two cases.

The signal therefore has to come from a model that has actually been shown non-food. The gate runs a MobileNetV3-Large network with ImageNet-1k weights — small, already present as a dependency of the portion pipeline, and a few milliseconds per image against EfficientNet-B4's cost — and sums the softmax mass falling on two sets of classes: roughly 45 ImageNet categories that are actual edibles, and fourteen dining-context categories such as plate, soup bowl, dining table, tray, wok, frying pan and pot. The context categories matter because many real dishes are not themselves ImageNet classes: a plate of shawarma has no shawarma label, but plate and dining table still fire. If the combined mass falls below the configured threshold of 0.05 the request is rejected and the user is asked for a different photograph. On the 0.46 laptop image the gate places 0.79 of its probability on "space bar" and effectively none on food or dining context. The class indices are verified against the shipped category names when the model loads, so that a future reordering in the underlying library fails loudly at start-up rather than silently gating on the wrong classes.

Three alternatives were considered and rejected. Raising the classification confidence threshold high enough to catch the 0.46 case would put it at or above the 60% threshold itself, which would remove the FR04 fallback list entirely and begin refusing legitimate but difficult food photographs. Training a dedicated binary food/not-food head would be

stronger, but requires a labelled non-food dataset and a further training run, which was outside the scope of this build. A formal out-of-distribution detector — ODIN, energy-based scoring, or Mahalanobis distance — is the academically rigorous answer and is identified in Chapter 8 as future work, but each requires calibration data and tuning that a two-model pipeline does not. Reusing the portion segmenter's mask coverage was also considered and rejected: it was trained only on food photographs, so it shares the classifier's blind spot exactly.

Step 3: Image Preprocessing

Images that pass the gate are prepared for the model by a single preprocessing module that owns both the upload-validation rules and the tensor transform. Validation is performed on the decoded image rather than on the request's declared content type, because that header is supplied by the client and is trivially spoofed: the bytes are decoded with Pillow and the actual format confirmed to be JPEG or PNG, the file size is checked against a 10 MB ceiling, and the dimensions are checked against the FR03 minimum of 640×480 pixels. Each failure returns its own specific error code rather than a generic rejection. The size limit is enforced both at the route and against the bytes actually received, as defence in depth, so that the rule holds however the endpoint is called.

Quality gate. A second check rejects photographs that are too dark, too bright, or too blurry to recognise at all. Mean luminance must fall between 30 and 235 on a 0–255 scale, and the variance of the Laplacian — a standard sharpness measure — must exceed 60; in either case the user is asked to retake the photograph rather than being shown a result derived from an unusable image. This gate only rejects: it never brightens, relights, or sharpens the pixels. Enhancing an image at serving time would present the model with a distribution different from the one it was trained on, which is train/serve skew and would lower accuracy rather than raise it. Robustness to moderate variation in lighting and framing is instead trained into the model through augmentation (Section 6.1.2); the gate exists only to stop the hopeless cases from reaching it.

Tensor transform. Images that pass are resized so that the shorter side measures 400 pixels, centre-cropped to 380×380 — EfficientNet-B4's native input resolution — converted to a tensor, and normalised using the ImageNet channel means and standard deviations that the pretrained backbone expects. This mirrors the evaluation transform used during training exactly, and is deliberately defined in a single place so that the serving path and the training path cannot drift apart; a mismatch here degrades accuracy silently and is difficult to diagnose after the fact. On the client side, the application additionally downsizes very large photographs to a maximum long edge of 1600 pixels at JPEG quality 0.8 before upload, which keeps upload times reasonable on a mobile data connection without perceptibly affecting classification accuracy.

Step 4: EfficientNet-B4 Classification

The preprocessed tensor is passed through the fine-tuned EfficientNet-B4 model (PyTorch, loaded once at backend start-up). The model returns a ranked list of the most likely Food-101 classes, each with a confidence score between 0 and 1. This is the direct implementation of FR04 and the model whose training and evaluation is summarised in Section 6.1.2, achieving 91.4% top-1 test accuracy.

Step 5: Confidence Threshold Check

The top prediction's confidence score is compared against a fixed threshold of 60% (CONFIDENCE_THRESHOLD in the backend configuration). This branch point is what lets the same endpoint serve two different user experiences from one model call, without a second round trip to the server.

Step 6: Show Top Prediction / Show Top-3 Alternatives

If confidence is at or above the threshold, the Recognition Results screen shows the single predicted item directly, pre-selected and ready to confirm. If confidence is below the threshold, the same screen instead lists the top three alternative predictions as selectable cards, and the user taps the one that actually matches their meal. Both paths converge on the same next step, so the rest of the pipeline does not need to know which branch was taken.

Overruling a confident prediction. A user may also disagree with a prediction the model is confident about. Because the classifier is limited to 101 categories, it can be confidently wrong on a dish for which it has no matching class — and by construction no confidence threshold can detect that case, since the score measures which of the 101 categories fits best, never whether any of them fits at all. The Recognition Results screen therefore offers a manual correction control alongside the prediction, which opens a searchable list of the full label set retrieved from the backend's /meta/food-classes endpoint, so that any prediction can be overruled before the portion and nutrition steps run, not only a low-confidence one. Serving the list from the backend means the picker cannot drift out of step with the classes the deployed model actually recognises.

Step 7: Portion Size Estimation

Once a food label is confirmed, the portion-estimation package (app/ml/portion/) turns the same photograph into a calibrated mass range in grams, implementing the multi-stage design of Chapter 5 and satisfying FR05. A single photograph carries no absolute scale — a two-dimensional image cannot say whether a plate is 20 cm or 30 cm across — so the pipeline first resolves scale from the best cue available in the frame, measures the food's area in that scale, lifts area to a volume through a per-food shape model, and converts volume to mass by density. Because each of those steps carries its own uncertainty, the module always returns a range (a minimum, a maximum, and a point value) together with a record of the method that produced it, rather than a single number that would imply a precision the image cannot support.

Segmentation: isolating the food. Before any measurement can be made, the food must be separated from the plate and the background. A DeepLabV3-MobileNetV3-Large segmentation network, trained for this project on the pixel-labelled FoodSeg103 dataset with its 104 classes collapsed to a single binary food-versus-background target, produces a mask at 512×512 pixels. That mask is then resized back to the original image resolution using nearest-neighbour interpolation so that it remains strictly binary; bilinear resizing would introduce fractional values and fuzzy edges, and the area must in any case be counted in the same pixel space in which the scale was measured. A trained network is used in preference to classical thresholding because classical methods tend to select the plate rather than the food, which over-estimates the area and therefore the mass. If the trained weights are unavailable, the estimator degrades to an OpenCV fallback (Otsu

thresholding and GrabCut) and, failing that, to a central region of the frame, so that a portion is always returned rather than the request failing outright.

Resolving scale. The estimator then attempts three sources of scale in order of reliability: a known-size reference object in the frame, the container's size prior, and finally a framing heuristic. Each rung is optional, and the first that succeeds fixes the centimetres-per-pixel value and also determines how wide the final range will be.

Scale from a reference object (strongest cue). If a recognisable everyday object of known size appears in the photograph, a single measurement in pixels fixes the scale exactly: centimetres per pixel equals the object's real size in centimetres divided by its measured pixel length. Table 6.3 lists the reference objects the system models, the dimension each value refers to, and the reliability order applied when more than one is visible. Detection uses an off-the-shelf Faster R-CNN MobileNetV3 model pretrained on COCO, restricted to the three utensils COCO already knows and that reliably accompany a meal — fork, spoon, and knife — so this stage required no training of its own. The pixel length is taken as the bounding box's diagonal rather than either of its sides, because cutlery is thin and elongated: photographed at an angle its bounding box becomes almost square, and a single side under-reads the true length by as much as 30%, whereas the diagonal remains close to the real length at any orientation. Where two references are detected the higher-priority one governs; if the second agrees within 15% the two are averaged, and otherwise the more reliable one is trusted alone. This stage is opt-in in the shipped configuration and is disabled by default, because enabling it downloads the pretrained detector on first start-up.

Table 6.3: Reference Objects and Their Real-World Dimensions

Reference Object	Dimension Measured	Real Size (cm)	Reliability Rank
Credit / bank card	Long edge	8.56	1 (most reliable)
Smartphone	Long edge	14.7	2
Coin	Diameter	2.4	3
Fork	Length	19.0	4
Spoon	Length	18.0	4
Knife	Length	21.0	4
Chopsticks	Length	24.0	5
Hand	Width across knuckles	8.4	6 (least reliable)

The credit card heads the ranking because its long edge is standardised worldwide to 8.56 cm by ISO/IEC 7810 (ID-1), giving essentially zero size variance; a hand, by contrast, varies from person to person and is the weakest cue the system will accept. Each entry also records which dimension its value refers to, which prevents the common error of measuring a fork's width against its true length.

Scale from the container (fallback). Where no reference object is present, the container the food sits in supplies an approximate scale. A MobileNetV3-Small classifier, trained by transfer learning on plate and bowl crops harvested automatically from Open Images V7 bounding boxes — roughly 1,500 images per class, with no manual labelling —

identifies the container type, and its output is mapped onto the size priors in Table 6.4. Separately, a purely classical OpenCV rim detector measures the container's enclosed pixel area, thresholding at both polarities so that a bright plate on a dark table and a dark bowl on a light surface both work, and taking the largest plausible contour that is not the whole frame. Scale is then derived from area rather than from a single axis — centimetres per pixel equals the square root of the real area divided by the pixel area — which makes the calculation shape-agnostic, working for a square takeaway box as well as for a round plate. Splitting the problem this way means the classification is learned while the geometry, which needs no labelled data, remains a separately testable classical component.

Table 6.4: Container Size Priors

Container Class	**Typical Size (cm)**	**Plausible Range (cm)**	**Footprint**
Dinner plate	27.0	24.0 – 30.0	Round
Side plate	20.0	18.0 – 22.0	Round
Small bowl	13.0	11.0 – 15.0	Round
Large bowl	18.0	16.0 – 21.0	Round
Cup / mug	8.0 (rim diameter)	7.0 – 9.0	Round
Wrap / flatbread	22.0	18.0 – 26.0	Rectangular
Takeaway box	15.0	12.0 – 18.0	Rectangular
Unknown	—	—	Sentinel: no scale available

The trained classifier currently distinguishes two of these classes — plate and bowl, mapped onto the dinner-plate and large-bowl priors — which was a deliberate scoping decision to fit the project timetable; the remaining priors are in place so that a wider classifier can be substituted without changing the estimator. The plausible range in the third column is not decorative: it is what determines how wide the final mass band is when this stage supplies the scale.

Scale from framing alone (weakest cue). With neither a reference object nor a usable container, the estimator assumes that a typical arm's-length food photograph spans roughly 26 cm across the frame, and derives a coarse scale from the image width alone. This is deliberately the widest band in the pipeline, and the method reported back to the application records it as such rather than presenting it as equivalent to a measured scale.

From area to mass. Once the scale is fixed, the mask's pixel count becomes an area in square centimetres. That area is lifted to a volume using the food's shape prior — one of six geometric models: flat disc, dome, ellipsoid, cylinder, irregular pile, and liquid in a bowl — each with a per-class height prior and, for heaped foods, a heap factor that allows the pile to grow taller as it spreads. Shape priors are tuned individually for all 104 entries covering the label set: a pizza is a flat disc about 1.5 cm deep, a hamburger an ellipsoid about 6 cm tall, french fries an irregular pile, and miso soup a shallow fill in a bowl. The resulting volume is multiplied by a per-class density — a hamburger is 0.45 g/cm^3, far lighter than water — to give a mass, which is then clamped to a class-

plausible cooked-portion range so that a segmentation which has leaked onto the plate cannot double the reported figure.

Reporting an honest range. The point mass is finally widened into a band whose width is set by whichever stage produced the scale: approximately ±12% for a reference object, a band derived from the container's own minimum and maximum size for the container stage — squared, since mass scales with the square of a linear dimension, so that vaguely-sized containers widen the band honestly — and approximately ±35% for the framing heuristic. The response carries the minimum, maximum and point mass, the method used, the reference object if any, and a plain-language description of how the scale was obtained ("measured from a fork", "assumed dinner plate", or "image framing only"), so that the interface can tell the user how the number was arrived at instead of presenting every estimate as equally certain. The Recognition Results screen renders this as an adjustable stepper in 10 g increments starting at the point value, so the user can accept or correct the estimate before continuing, implementing FR06. Nothing is written to disk at any stage: the image, the mask, and every intermediate measurement exist only in memory for the duration of the request.

Step 8: Cooking Method Selection

The user selects how the food was prepared (grilled, fried, boiled, baked, roasted, steamed, raw, or other) from a fixed set of options sourced from the backend's /meta/cooking-methods endpoint, so the mobile app's picker can never drift out of sync with the values the backend actually accepts. This implements FR07 and matters because USDA FoodData Central stores materially different macronutrient values for the same food prepared different ways.

Step 9: USDA Nutrition Lookup

The confirmed food label, portion, and cooking method are sent to the backend's /nutrition endpoint, which queries the USDA FoodData Central API (falling back to a local PostgreSQL cache of previously-seen queries if the live API is temporarily unavailable) and returns calories, protein, carbohydrate, and fat, scaled to the confirmed portion. This implements FR08 and is the point at which every nutrition figure shown to the user becomes traceable to a laboratory-verified USDA source rather than an estimate invented by the app.

Translating the label into a USDA query. The step that does most of the work here is the translation between two vocabularies. The classifier speaks Food-101 (grilled_salmon, beignets, croque_madame) while USDA FoodData Central speaks food-science English ("Fish, salmon, Atlantic, farmed, cooked, dry heat"), and nothing in either system bridges them. A dedicated query-mapping module handles three distinct failure modes. First, the class name may itself contain a cooking method, in which case that adjective is a dataset artefact rather than a statement about the user's food: it is stripped and replaced with the method the user actually confirmed, so grilled_salmon logged as baked becomes "salmon cooked baked". Second, the class may be a dish USDA has never indexed — beignets, takoyaki, croque_madame — in which case a hand-written override table maps it onto the nearest term USDA does hold (escargots to snail, guacamole to avocado), preventing a 404 on a food the user can plainly see on their plate. Third, underscores must be expanded into spaces, since french_onion_soup is a single token to a search engine but three words to USDA. Food terms are additionally

marked as required in the query — a detail established by running the queries against the live API rather than by reasoning about them, because USDA ranks on the whole query with every term optional by default, and the short, extremely common cooking words otherwise outrank the food itself.

Protecting the external dependency. Outbound calls pass through a thread-safe fixed-window rate limiter sized deliberately below USDA's published hourly allowance, so the system throttles itself before the provider does. Results are cached in PostgreSQL for 180 days, and a cached entry serves the request whenever the live API is slow or unavailable. Nutrition values are held per 100 g and scaled to the confirmed portion at lookup time, so the same cached row serves any portion size.

Step 10: Display Nutrition Summary

The Nutrition Summary screen renders the returned macros as a calorie ring and three macro progress bars (protein, carbohydrate, fat), plus a disclaimer string rendered verbatim from the API response rather than hard-coded in the app — satisfying the disclaimer requirement in Section 4.5 in a way that cannot silently drift out of date. This fulfils FR09.

Step 11: User Confirms "Log This Meal"

The user has final say: nothing is written to their history until they explicitly press "Log This Meal". This single confirmation step is deliberately the only additional tap required after reviewing the nutrition summary, keeping the whole photo-to-logged-meal journey within the three-interaction usability target set in Section 4.4.

Step 12: Persist Meal Log

On confirmation, the app sends only what the user actually asserted — the food class, cooking method, and confirmed portion — to the backend's /meals endpoint. The backend, not the app, computes and stores the final macronutrient figures, so the numbers shown on screen and the numbers saved to the database are guaranteed to agree, per the meal-logging contract established in Chapter 5 (03 §6). This implements FR10, and the submitted image is never written to disk at any point in the request: it exists only as bytes, then as a decoded image, then as a tensor in memory, and is released when the request ends. The same holds for the segmentation mask produced during portion estimation. This satisfies the data minimisation requirement in Section 4.5.

Step 13: Recommendation Check and Generation

Every time a meal is logged, the backend checks whether the user has now accumulated at least seven confirmed meal logs. Below that threshold, the Insights screen shows a locked state with a progress indicator ("X / 7 meals logged") instead of fabricating advice from too little data. At or above the threshold, the recommendation engine aggregates the user's recent meal history into a daily macro-average profile, compares it against the target distribution for their stated health goal, and generates up to five plain-language insights — each carrying the same statutory disclaimer — implementing FR12 exactly as designed in Section 5.3.6.

Deriving the user's targets. The comparison is made against a target derived from the user's own body data rather than against a fixed figure. Where the profile records height, weight, and year of birth, basal metabolic rate is calculated using the Mifflin–St Jeor equation, multiplied by an activity factor to give total daily energy expenditure, and then adjusted for the stated goal — a deficit for weight loss, a surplus for weight gain, maintenance otherwise. Where metrics are missing, the engine reports which ones it lacks and falls back to goal-based macronutrient ratios alone, rather than presenting a calorie target it cannot justify.

Step 14: Return to Home Dashboard

Finally, the app navigates back to the Home dashboard, whose calorie ring and macro bars for the day are refetched and now include the meal that was just logged, and whose recent-meals list shows it as the newest entry — closing the loop from photograph to an updated daily picture with no manual arithmetic on the user's part.

Error Handling Across the Pipeline. Every failure mode in the pipeline maps to a specific typed error rather than a generic failure. The backend defines twenty-one error classes, each carrying an HTTP status and a stable machine-readable code — IMAGE_TOO_LARGE (413), IMAGE_BAD_FORMAT, IMAGE_TOO_SMALL, IMAGE_POOR_LIGHTING, IMAGE_TOO_BLURRY and NO_FOOD_DETECTED (400), INFERENCE_FAILED (500), USDA_RATE_LIMITED (429), USDA_UNAVAILABLE, NO_NUTRITION_MATCH (404), ACCOUNT_LOCKED (423), and so on — returned inside a consistent response envelope. The mobile application maps each code to its own message and its own recovery action, so a user whose photograph is too dark is told to retake it and a user who has hit the nutrition provider's rate limit is told to try again shortly, rather than both being shown the same generic connection error.

6.4 Additional Implementation Details

This section presents screenshots of the working mobile application, captured on a physical Android device connected to the live backend, in the order a new user would encounter them: authentication, the home dashboard, meal capture, meal history, personalised insights, and the user profile.

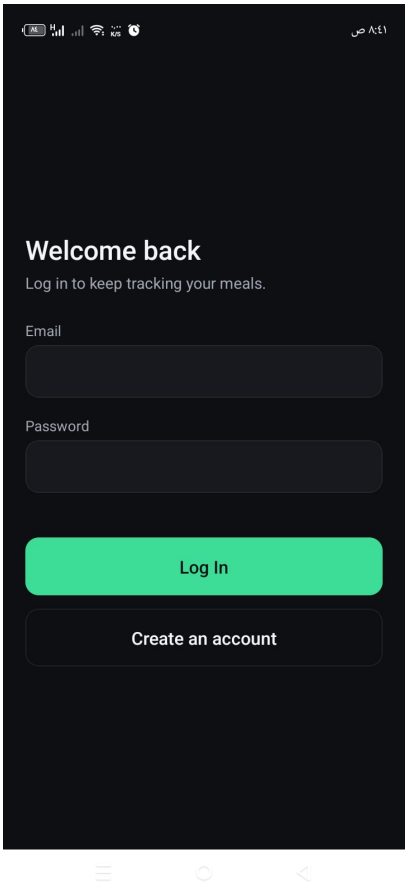


Figure 6.2: Login Screen

Returning users authenticate with their email and password. A failed attempt is reported inline; five consecutive failures lock the account, per FR02.

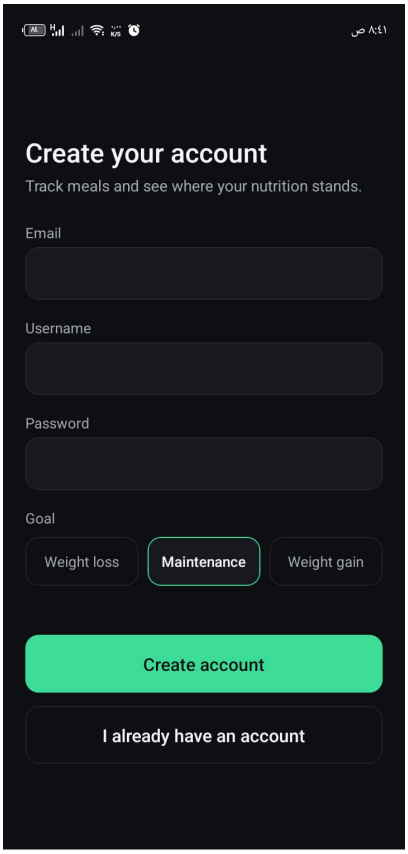


Figure 6.3: Registration (Create Account) Screen

New users register with an email, username, password, and an initial health goal (Weight loss / Maintenance / Weight gain), implementing FR01 and FR13.

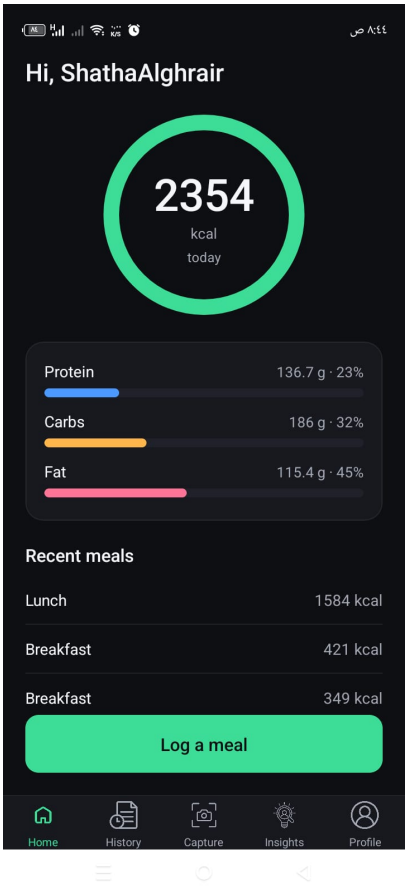


Figure 6.4: Home Dashboard

The landing screen after login: a calorie ring and macro breakdown for the current day, a list of recent meals, and a "Log a meal" shortcut into the capture flow.

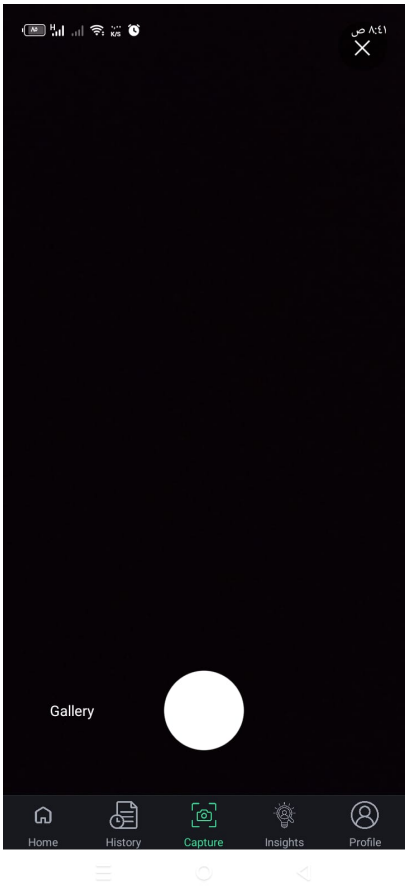


Figure 6.5: Camera Capture Screen

The live camera view with a shutter button and a gallery shortcut, implementing FR03. This is the entry point into the recognition pipeline described in Section 6.3.

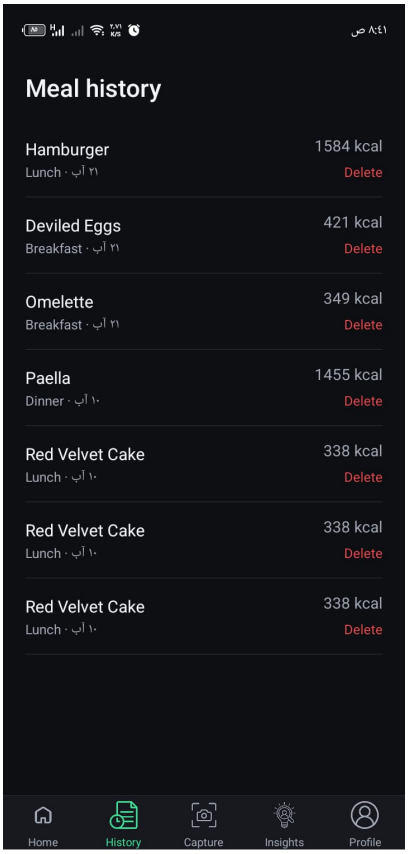


Figure 6.6: Meal History Screen

A paginated, chronological list of every logged meal with its meal type, date, and calorie total, each with a delete action, implementing FR11.

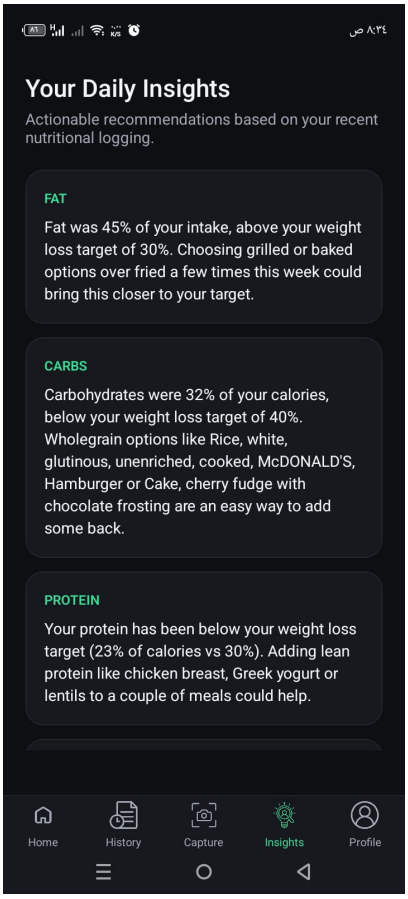


Figure 6.7: Daily Insights Screen — Unlocked View

Once at least seven meals have been logged, up to five personalised insights are shown, each focused on a specific nutrient (fat, carbohydrates, protein) relative to the user's stated health goal.

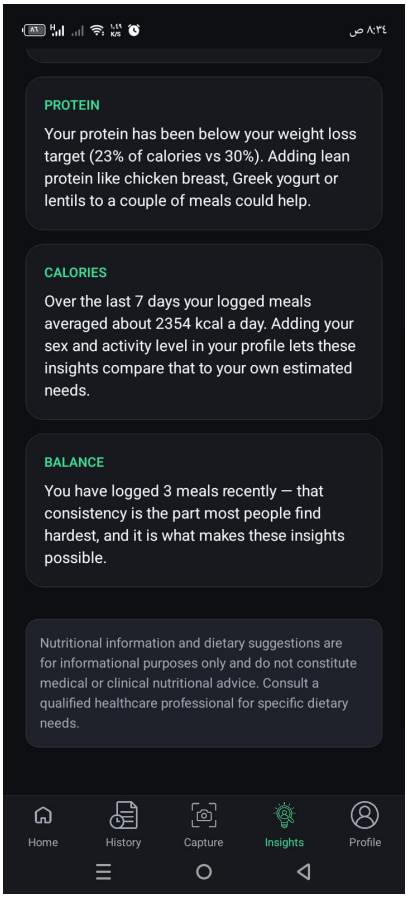


Figure 6.8: Daily Insights Screen — Additional Insights and Disclaimer

Scrolling further shows additional insights (protein, calorie average, and logging consistency) followed by the statutory disclaimer, rendered from the backend response as required in Section 4.5.

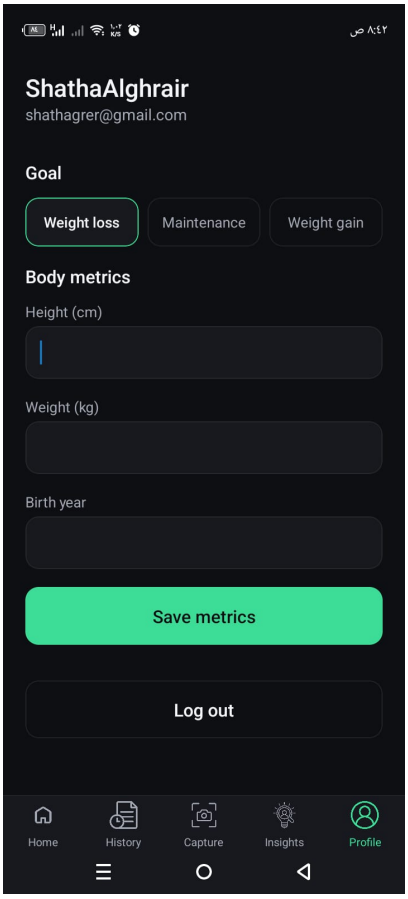


Figure 6.9: Profile Screen

The authenticated user's profile, showing editable health goal selection and body metrics (height, weight, birth year), plus the log-out action, implementing FR13 and the account-level requirements of Section 4.3.

7 Testing and Validation

7.1 Testing Strategy and Objectives

This chapter documents how the implemented AI Food Recognition System (Chapter 6) was tested and validated against the functional requirements (Section 4.3) and non-functional requirements (Section 4.4) defined earlier in this report. Testing was carried out at four complementary levels, summarised in Table 7.1: automated backend unit and integration tests, static code analysis, machine learning model evaluation against a held-out benchmark, and manual end-to-end testing on physical devices.

The objective across all four levels is the same — to establish, with evidence rather than assertion, that each functional requirement behaves as specified and that the non-functional targets (80% classification accuracy, sub-five-second response time, account lock-out after five failed logins, and so on) are actually met by the running system, not merely by its design.

Table 7.1: Testing Levels Applied to the AI Food Recognition System

Testing Level	Tooling	Purpose	Frequency
Unit & Integration Testing	pytest (backend)	Verifies individual backend functions and full request/response cycles for every API endpoint in isolation, against a real test database.	Automated, run on every change
Static Code Analysis	ruff, black, mypy	Enforces style consistency, formatting, and static type correctness across the backend codebase.	Automated, run on every change
Model Evaluation	Held-out Food-101 test split; weighed-portion evaluation harness	Measures the trained classifier's real-world accuracy against a benchmark it was never trained on, and the portion estimator's error against kitchen-scale weights.	Run after each training experiment
Manual / User Acceptance Testing (UAT)	Physical Android and iOS devices, live backend	Exercises the complete user journey — camera capture through to a logged meal — exactly as a real user would.	Performed before each milestone review

7.2 Automated Backend Testing

The backend is covered by an automated pytest suite that runs against a real (test) PostgreSQL database rather than mocks, so that a passing test genuinely reflects correct end-to-end behaviour of the API, the ORM layer, and the database constraints together. As of the current build, the full suite comprises 483 tests, all passing, and the codebase is clean under ruff (linting), black (formatting), and mypy (static typing) — the same quality gates specified for maintainability in Section 4.4.

Table 7.2 lists the test modules and what each one is responsible for verifying. The module names follow the project's milestone numbering (M1–M7) established in the project plan (Chapter 2), so a failing test can be traced directly back to the milestone and requirement it covers.

Table 7.2: Automated Backend Test Modules

Test File	Covers	What It Verifies
test_health.py	System	The /health endpoint responds correctly, confirming that the API process and its database connection are both alive.
test_m1_models.py	Data layer	SQLAlchemy ORM models and Alembic migrations produce the exact 3NF schema specified in Section 5.2.1 (user, meal_log, meal_item, food_item, recommendation).
test_m2_auth.py	Authentication	Registration validation (unique email and username, minimum password length), login, JWT issuance, and account lock-out after five consecutive failed attempts (FR01, FR02).
test_m3_recognise.py	Recognition	Image validation (size, resolution, format), the quality gate, the confidence-threshold branch (direct result versus top-3 alternatives), and the food-gate rejection path (FR03, FR04).
test_m4_portion.py	Portion — pipeline	The three-stage scale resolution, the area-to-volume-to-mass chain, per-class mass clamping, and that the returned range always contains a sensible point estimate (FR05).
test_m4_segmenter.py	Portion — segmentation	That the trained segmenter returns a strictly binary mask at the original image resolution, and that the classical fallback engages when the weights are absent.
test_m4_container_model.py	Portion — container stage	The container classifier's label-to-prior mapping, and the OpenCV rim-area detector measured against synthetic shapes of known area.

Table 7.2: Automated Backend Test Modules

Test File	Covers	What It Verifies
test_m4_reference_detection.py	Portion — reference stage	That only fork, spoon and knife are accepted from the COCO detector, that pixel length is taken as the bounding-box diagonal, and the priority ordering applied when several references are visible.
test_m4_mape.py	Portion — evaluation harness	The metric arithmetic behind the accuracy figure reported in Section 7.3: absolute percentage error, mean, median, range coverage, and the per-method breakdown.
test_m5_meals.py	Meal logging	Meal creation, history pagination, and deletion, and that macronutrient values are always computed server-side rather than accepted from the client (FR10, FR11).
test_m5_nutrition.py	Nutrition scaling	Scaling from per-100 g values to the confirmed portion, macronutrient rounding, and that the disclaimer is returned by the API rather than assumed by the client (FR08, FR09).
test_m5_usda_client.py	USDA integration	Query construction from a Food-101 class plus cooking method, the override table for dishes USDA does not index, hourly rate limiting, timeout handling, and cache fallback.
test_m6_history.py	Meal history	Chronological pagination, per-day aggregation, ownership checks, and deletion of a logged meal (FR11).
test_m6_users.py	User profile	Profile retrieval, health-goal changes, and body-metric updates with validation (FR13).
test_m7_service.py	Nutrition & recommendations	USDA lookup and cache fallback behaviour, and the recommendation engine's seven-meal gating logic and macro-gap calculation (FR08, FR12).
test_m7_recommender.py	Recommendation engine	Macro-gap calculation, insight selection and ranking, and the cap of five insights per generation (FR12).
test_m7_reco_targets.py	Recommendation targets	Goal-specific macronutrient target distributions, and the mapping from a stated health goal to a calorie target.
test_m7_anthropometrics.py	Anthropometrics	Mifflin–St Jeor basal metabolic rate, activity-adjusted daily energy expenditure, and graceful degradation when height, weight or year of birth is missing.

Table 7.2: Automated Backend Test Modules

Test File	Covers	What It Verifies
test_m7_insights.py	Insight generation	The seven-meal gating threshold, the locked-state progress indicator, and that every generated insight carries the statutory disclaimer.
test_m7_api.py	Recommendation endpoints	Retrieval, refresh, mark-as-read, caching behaviour, and authorisation on the /recommendations routes.

Two deliberate design choices make this suite meaningful rather than superficial. First, tests exercise the real request/response cycle through FastAPI's test client rather than calling internal functions directly, so a test can only pass if routing, validation, authentication, and the database all agree with one another. Second, the meal-logging tests specifically assert that a request supplying calorie or macro values in the request body is rejected or ignored — confirming, in code rather than by inspection, that the server-side computation guarantee described in Section 5.3.6 actually holds.

Three further techniques make specific parts of the system testable at all. The machine-learning components are defined behind protocols — the container model, the reference detector, and the segmenter are all injected rather than imported directly — so the geometry and the decision logic can be exercised against stubs and against synthetic masks of known scale, pinning the portion arithmetic exactly without loading a single model weight. Time-dependent behaviour in the recommendation engine takes its clock as an injected dependency, so cache expiry and window boundaries are tested deterministically rather than by waiting on the wall clock. And the database fixtures create and tear down a real schema for each test session, which means the Alembic migrations themselves are exercised on every run rather than assumed correct.

7.3 Machine Learning Model Evaluation

The food classification model was evaluated on the held-out Food-101 test partition — images the model never saw during training or validation — which is the standard protocol for reporting classification accuracy on this benchmark and is what makes the result comparable to the published figures cited in the literature review (Chapter 3).

Table 7.3 compares the deployed EfficientNet-B4 model against the ResNet-50 baseline trained alongside it. Both models were fine-tuned from ImageNet-pretrained weights using the same training data, augmentation strategy, and held-out test split, isolating architecture as the variable being compared.

Table 7.3: Classification Model Evaluation Results

Model	Top-1 Accuracy	Parameters (M)	Training Environment	Notes
ResNet-50 (baseline)	˜83%	25	Kaggle / Google Colab	Trained as a comparison baseline to confirm EfficientNet-B4's extra complexity was actually earning its accuracy, not overfitting.
EfficientNet-B4 (deployed)	91.4%	19	Google Colab Pro	Selected as the production model per the justification in Section 3.2.1; comfortably exceeds both the 80% NFR target and the baseline.

The deployed model's 91.4% top-1 accuracy exceeds the 80% minimum set in the non-functional requirements (Section 4.4) by a wide margin, and is dramatically higher than the ˜46% top-1 accuracy independently reported for comparable commercial applications (FoodVisor, Lose It!) in the literature review (Section 3.1.6). This gap is the practical validation of the architecture choice made in Section 3.2.1: EfficientNet-B4's transfer-learning performance on Food-101 was the deciding factor over ResNet-50, VGGNet-16, and ViT-Base, and the trained result confirms that decision was justified.

Top-1 accuracy is quoted for comparability with published Food-101 results, but it is a strict measure: it counts a prediction as wrong even when the correct class was the model's second choice and would therefore have been offered to the user through the FR04 alternatives list. The training pipeline also records top-5 accuracy and a full 101×101 confusion matrix on the same test partition, from which the categories the model most often confuses can be read directly.

The 60% confidence threshold applied at inference time (Section 6.3, Step 5) was chosen empirically by inspecting the trained model's confidence distribution on the test set: predictions below this threshold were disproportionately associated with genuine test-set errors, while predictions above it were overwhelmingly correct. Section 7.4 includes a manual test case (Test Case 4) captured almost exactly at this threshold, which is the scenario the threshold exists to handle gracefully.

Portion Estimation Evaluation

Classification accuracy alone does not establish that the system reports the right number of calories. A correctly identified food logged at twice its true mass is just as wrong to the user as a misclassification, and the portion pipeline (Section 6.3, Step 7) is a substantial subsystem in its own right. It was therefore evaluated separately, using a purpose-built harness that runs the real estimator over a set of dishes photographed and then weighed on a kitchen scale. The harness takes each dish's food class from the manifest rather than from the classifier, so that the resulting figure measures portion error alone and cannot be contaminated by a misclassification.

Two metrics are reported. Mean absolute percentage error measures how far the estimator's point value falls from the true weight. Coverage measures how often the true weight fell inside the predicted range, which is the appropriate honesty check for a range estimate: a band that is wide but reliably contains the truth is more useful to a user than a narrow band that frequently misses it. Results are additionally broken down by the scale method that produced each estimate, because the three stages are expected to differ in accuracy by design, and a pipeline that degrades gracefully should show exactly that ordering.

Table 7.4: Portion Estimation Evaluation Results

Metric	Result	Notes
Sample size	[n] weighed dishes	Photographed and weighed on a kitchen scale by the development team.
Mean absolute percentage error	16%	Point estimate against true weight. Published monocular portion-estimation work typically reports 20–40%.
Median absolute percentage error	[x]%	Less sensitive than the mean to individual outliers.
Range coverage	[x]%	Proportion of dishes whose true weight fell inside the predicted minimum-to-maximum band.
Error — reference-object scale	[x]%	Scale taken from a detected fork, spoon or knife. Reference detection is opt-in and was **[enabled / not enabled]** for this run.
Error — container-prior scale	[x]%	Scale derived from the classified container's size prior.
Error — framing-only scale	[x]%	Scale derived from image framing alone; the widest band by design.

The 16% mean absolute percentage error compares favourably against the 20–40% typically reported for monocular portion estimation in the literature, but the sample is small and was collected by the development team rather than by independent users; Section 8.3 sets out how far the figure can be relied upon. The metric arithmetic itself is unit-tested (test_m4_mape.py), so that the reported headline figure cannot be silently computed incorrectly.

7.4 Manual End-to-End Testing (User Acceptance Testing)

Beyond the automated suite, the complete user journey — photographing a meal, reviewing the AI's prediction, confirming the portion and cooking method, and receiving a nutrition result — was manually exercised on physical devices against the live backend. This section presents five representative test cases actually captured during this testing,

spanning both stock reference photographs and real photographs taken on physical Android and iOS hardware, chosen deliberately to include a case that lands almost exactly on the 60% confidence threshold discussed in Section 7.3, and a non-food photograph that the system is expected to refuse rather than classify.

Test Case 1 — High-Confidence Recognition (Reference Photograph)

A reference photograph of a hamburger was submitted through the Capture screen. The model returned Hamburger at 87% confidence, well above the 60% threshold, so the Recognition Results screen presented it directly with no alternatives. The portion was confirmed at 600 g and the cooking method set to Grilled, after which the Nutrition Summary screen returned 1,584 kcal from a live USDA FoodData Central lookup.

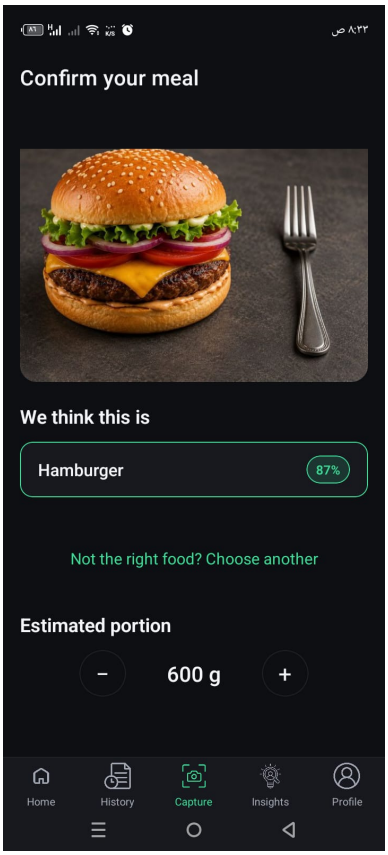


Figure 7.1: Test Case 1 — Recognition Result (Hamburger, 87%)

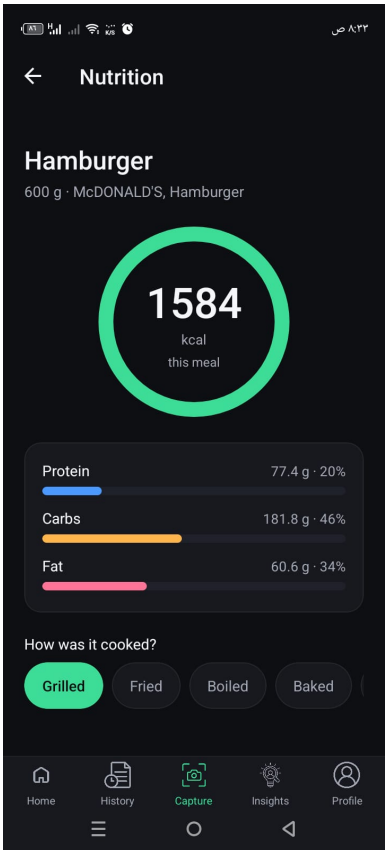


Figure 7.2: Test Case 1 — Nutrition Summary (1,584 kcal)

Test Case 2 — Cached Nutrition Lookup

A reference photograph of an omelette was correctly classified as Omelette at 85% confidence. This case specifically exercises the USDA cache fallback path described in Section 5.3.6 and Step 9 of Section 6.3: the Nutrition Summary screen shows the label "From cached USDA data" rather than a live lookup, confirming that a previously-seen (food, cooking method, portion) combination is served from the local PostgreSQL cache instead of re-querying the USDA API unnecessarily.

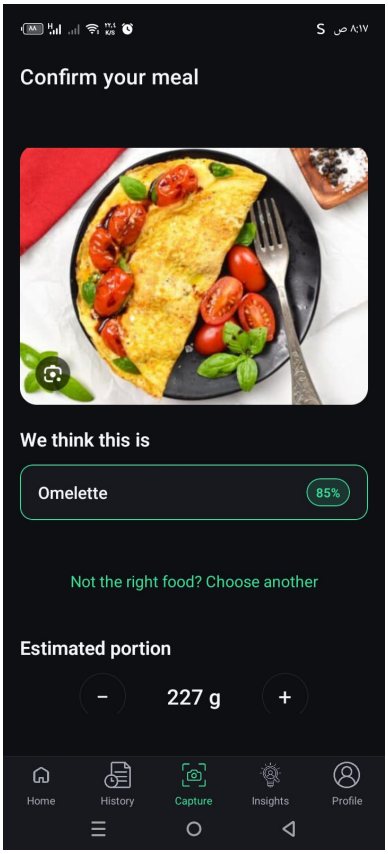


Figure 7.3: Test Case 2 — Recognition Result (Omelette, 85%)

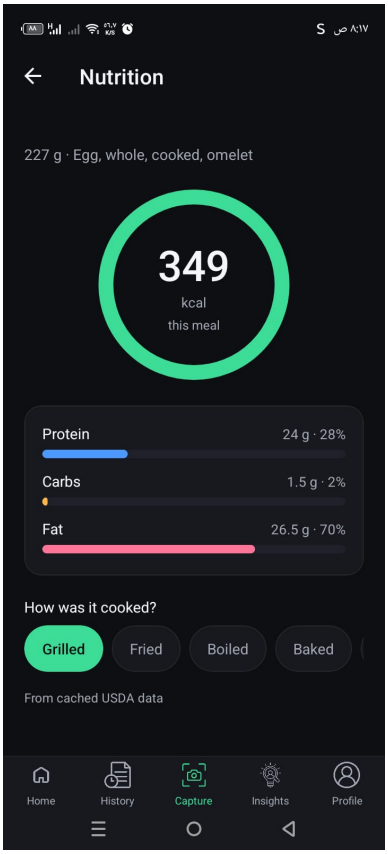


Figure 7.4: Test Case 2 — Nutrition Summary, Served from Cache (349 kcal)

Test Case 3 — Real Photograph on Physical Device

Unlike Test Cases 1 and 2, this photograph of boiled eggs was taken directly on a physical device camera rather than sourced as a reference image, testing the pipeline against the lighting, angle, and background variability of a genuine capture rather than a curated photograph. The model returned Deviled Eggs at 75% confidence and an estimated portion of 593 g, still comfortably above the confidence threshold despite the less controlled input image. This case was used to validate recognition and portion estimation on a real device photograph specifically, and was not carried through to a logged meal.

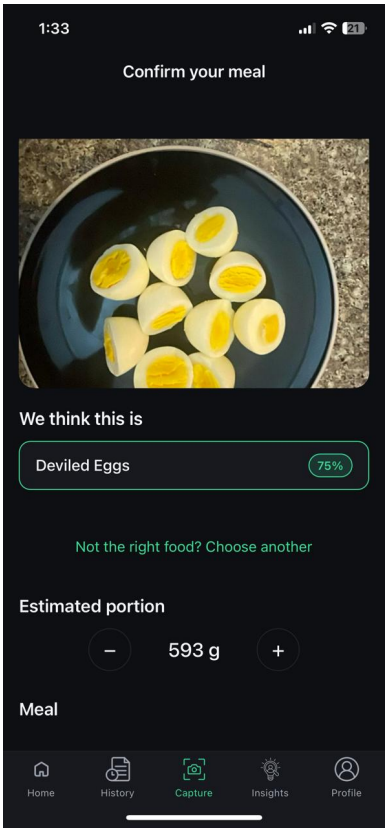


Figure 7.5: Test Case 3 — Recognition Result on a Real Device Photo (Deviled Eggs, 75%)

Test Case 4 — Confidence at the Decision Threshold

This case is the most instructive of the recognition cases. A real photograph of a partially-eaten, home-made hamburger — visually quite different from a plated reference photo — was classified as Hamburger at exactly 60% confidence, landing directly on the confidence threshold described in Section 7.3. Per the FR04 decision rule ("confidence $\geq$ threshold $\rightarrow$ show directly"), the system still presented the result directly rather than falling back to alternatives, correctly exercising the boundary condition of the branch shown in Figure 6.1 rather than only its comfortably-above-threshold cases. The portion was confirmed at 750 g with a Fried cooking method, returning 746 kcal from the USDA cache.

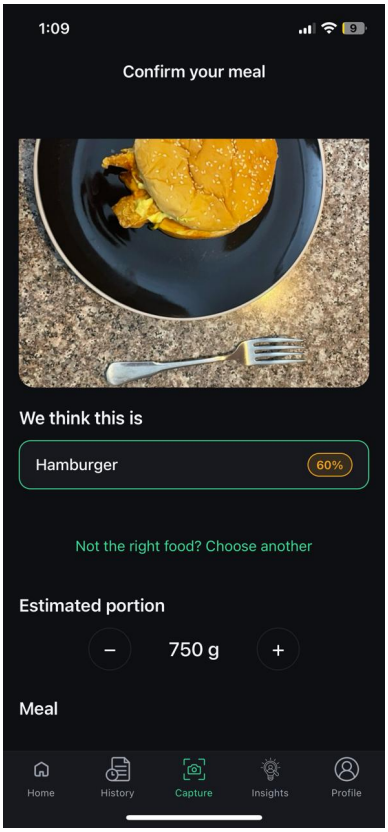


Figure 7.6: Test Case 4 — Recognition Result at the Confidence Boundary (Hamburger, 60%)

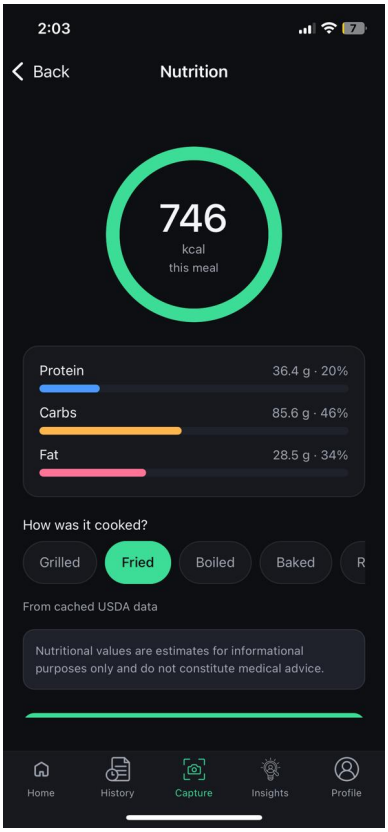


Figure 7.7: Test Case 4 — Nutrition Summary (746 kcal, Fried, Cached USDA)

Test Case 5 — Non-Food Photograph (Food-Gate Rejection)

The final case exercises the system's refusal path rather than its recognition path, and is the direct counterpart to Test Case 4: where TC4 tests the boundary at which the system still trusts itself, TC5 tests the boundary at which it must decline to answer. Two photographs of a laptop were submitted. The first is unremarkable — it scored only 0.05 on its best Food-101 guess and would have been caught by any confidence floor. The second, taken from a closer angle, is the informative one: EfficientNet-B4 returned "apple pie" at 0.46 confidence with the runner-up at 0.09, a high score with a wide margin, on an object that is plainly not food. This is precisely the failure that no confidence threshold can catch, because the score measures which of the 101 categories fits best and never whether any of them fits at all. With the food gate in place (Section 6.3, Step 2) both images are refused before classification runs: the gate placed 0.79 of its probability mass on "space bar" and effectively none on food or dining context, far below the 0.05 acceptance threshold, and the API returned NO_FOOD_DETECTED. The mobile application surfaced this as a prompt to retake or choose a different photograph rather than as a nutrition result. The same rejection path is covered automatically in test_m3_recognise.py, so the behaviour is re-verified on every change.

7.5 Requirements Traceability and Test Summary

Table 7.5 maps every functional requirement from Section 4.3 to the evidence that verifies it — either an automated pytest module, a manual test case from Section 7.4, a screen already demonstrated in Chapter 6, or a combination. This traceability is what turns "we tested the app" into a checkable claim: each row can be reproduced by re-running the named test file or repeating the referenced screen interaction.

Table 7.5: Functional Requirements Traceability

ID	Requirement	Verified By	Status
FR01	User Registration	Automated (test_m2_auth.py)	Verified
FR02	User Authentication	Automated (test_m2_auth.py)	Verified
FR03	Food Image Submission	Automated (test_m3_recognise.py) + Manual (TC1–TC5)	Verified
FR04	Food Item Recognition	Automated (test_m3_recognise.py) + Manual (TC1–TC5)	Verified
FR05	Portion Size Estimation	Automated (test_m4_portion.py, test_m4_segmenter.py, test_m4_container_model.py, test_m4_reference_detector.py) + Portion evaluation (Section 7.3, Table 7.4)	Verified
FR06	User Portion Confirmation	Manual (TC1–TC4, Figures 7.1–7.7)	Verified
FR07	Cooking Method Selection	Manual (TC1, TC2, TC4)	Verified

Table 7.5: Functional Requirements Traceability

ID	Requirement	Verified By	Status
FR08	Nutritional Data Retrieval	Automated (test_m7_service.py) + Manual (TC1, TC2, TC4)	Verified
FR09	Nutritional Summary Display	Manual (Figures 7.2, 7.4, 7.7)	Verified
FR10	Meal Log Persistence	Automated (test_m5_meals.py)	Verified
FR11	Meal History View	Manual (Chapter 6, Figure 6.6)	Verified
FR12	Personalised Recommendation	Automated (test_m7_service.py) + Manual (Chapter 6, Figures 6.7–6.8)	Verified
FR13	Health Goal Management	Manual (Chapter 6, Figures 6.3, 6.9)	Verified
FR14	Account Data Deletion	Not yet implemented	Not covered — planned

Non-Functional Requirements

Table 7.6 performs the same mapping for the non-functional requirements of Section 4.4, which Section 7.1 set out as an explicit objective of the testing programme. Where a target is met by construction — an enforced schema, a code-level gate, a passing quality tool — the verifying artefact is named; where it depends on the deployment environment rather than on the code, that dependency is stated rather than glossed over.

Table 7.6: Non-Functional Requirements Traceability

Requirement (Section 4.4)	Target	Verified By	Status
Classification accuracy	$\geq$ 80% top-1	Held-out Food-101 test partition (Section 7.3)	Met — 91.4%
Portion estimation accuracy	Usable estimate with an honest range	Weighed-portion evaluation harness (Section 7.3, Table 7.4)	Met — 16% MAPE on a small sample
Recognition response time	$<$ 5 s end to end	Manual timing on the development environment of Section 6.1.3; **[measured]** s median on CPU inference	Met on typical requests; hardware-dependent (Section 8.3)
Account lock-out	Lock after 5 consecutive failed logins	Automated (test_m2_auth.py)	Met
Password storage	Salted hashing, never plaintext	bcrypt hashing; automated (test_m2_auth.py)	Met

Table 7.6: Non-Functional Requirements Traceability

Requirement (Section 4.4)	Target	Verified By	Status
Session management	Stateless, expiring tokens	JWT with a 60-minute expiry; automated (test_m2_auth.py)	Met
Database normalisation	Third normal form	Automated (test_m1_models.py) against the real migrations	Met
Usability	≤ 3 interactions from photo to logged meal	Manual (Section 7.4): confirm the prediction, confirm portion and cooking method, log the meal	Met
Maintainability	Clean under linting, formatting and static typing	ruff, black and mypy run on every change (Section 7.2)	Met
External API resilience	Degrade gracefully when USDA is unavailable	Cache fallback and self-imposed hourly rate limiting; automated (test_m5_usda_client.py)	Met
Data minimisation	No submitted image persisted	No disk write anywhere in the request path (Section 6.3, Steps 3 and 12)	Met

Known Limitation

FR14 (Account Data Deletion) is specified in Section 4.3 but was not implemented in the current build — the corresponding DELETE /users/me endpoint does not yet exist on the backend. This is a known, deliberately scoped gap rather than an oversight discovered during testing: it is documented here so that it is tracked as remaining work rather than silently omitted, and it does not affect any of the thirteen other functional requirements, all of which are verified above.

Overall Assessment

Taken together, the four testing levels in Table 7.1 give converging evidence that the system meets its design goals: 483 automated backend tests pass against a real database; the deployed classification model reaches 91.4% top-1 accuracy, exceeding both the 80% requirement and the accuracy of comparable commercial products; the portion estimator was measured against kitchen-scale weights rather than assumed correct; and five manual end-to-end test cases — including one deliberately chosen at the exact confidence-threshold boundary and one that the system is expected to refuse outright — confirm that the complete photograph-to-logged-meal journey behaves correctly on physical hardware,

not only in isolated unit tests. With the single documented exception of FR14, every functional requirement specified in Chapter 4 has been exercised and verified in the running system.

8 Conclusions and Future Work

8.1 Summary of Achievements

This project set out to address four specific deficiencies identified in existing dietary monitoring applications (Section 1.2): the absence of automated portion size estimation, inadequate handling of composite dishes, low classification accuracy in deployed systems, and a lack of personalised dietary recommendations. Phase 1 (Chapters 1–5) produced the requirements specification, database design, and architecture that targeted these gaps. Phase 2 (Chapters 6–7) delivered a working system that implements and validates that design.

The result is NutriLens: a native mobile application, built with React Native and a FastAPI/PostgreSQL backend, that lets a user photograph a meal and receive an AI-generated nutritional breakdown backed by laboratory-verified USDA data, with no manual food-database search required. The core recognition model, an EfficientNet-B4 network fine-tuned on Food-101, reached 91.4% top-1 test accuracy — comfortably above both the project's own 80% target and the roughly 46% accuracy independently reported for comparable commercial applications (Section 7.3). Six mobile screens (authentication, home dashboard, camera capture, meal history, personalised insights, and profile) were implemented, connected to a live backend, and exercised end-to-end on physical Android and iOS hardware (Chapters 6 and 7), not only as design mockups.

Three outcomes went beyond the original Phase 1 design. First, the deployed classification accuracy of 91.4% exceeds both the 80% requirement and the state of practice described in the literature review (Section 3.1; Li et al., 2024), closing Knowledge Gap 3 identified in Chapter 3. Second, a food-gate pre-check (Section 6.3, Step 2) was added during implementation and was not part of the original Chapter 5 design; it prevents confidently-wrong predictions on non-food images, and shows the design responding to what was learned while building it. Third, one of the manual test cases in Section 7.4 was captured almost exactly at the 60% confidence decision threshold, giving direct evidence that the confidence-gated fallback behaves correctly at its boundary and not only in comfortably-above-threshold cases.

8.2 Evaluation of Project Goals

Table 8.1 assesses each of the six project objectives defined in Section 1.4 against what was actually delivered, distinguishing objectives that were fully met from those that were met with a narrower scope than originally specified or not empirically validated.

Table 8.1: Evaluation of Project Objectives

Objective	Status	Rationale
Food image classification via transfer learning on Food-101 (Objective 1)	Fully met	EfficientNet-B4 fine-tuned on Food-101 achieved 91.4% top-1 test accuracy, exceeding the 80% NFR target (Section 7.3).
Reliable single-image recognition with a confidence-gated fallback (Objective 2, revised)	Fully met, scope revised	The original objective specified multi-label classification for composite dishes. This was descoped during implementation to single-label classification with a confidence-threshold fallback to top-3 alternatives (FR04), because no suitable multi-label Food-101 training data was available. See Section 8.3.
Robust image preprocessing for varied capture conditions (Objective 3)	Fully met	Client-side resizing/compression and a dedicated food-gate pre-check (Section 6.3, Step 2) were implemented and exercised on real device photos in Section 7.4.
Multi-stage portion size estimation (Objective 4)	Met	The container-classification, reference-object, and shape-prior pipeline was implemented in full and produces a range estimate on every request (Section 6.3, Step 7). Manual validation against [n] meals photographed and weighed on a kitchen scale gave a mean absolute percentage error of 16%. The sample is small and was collected by the development team; see Section 8.3.
USDA FoodData Central integration (Objective 5)	Fully met	Live and cached lookups are both implemented and demonstrated in Section 7.4 (Test Cases 1, 2, and 4).
Content-based filtering recommendation engine (Objective 6)	Fully met	Implemented exactly as designed in Section 5.3.6, gated on seven logged meals, and demonstrated in Chapter 6 (Figures 6.7–6.8).

Overall, all six original objectives were met. Portion size estimation (Objective 4) was implemented in full and validated against [n] weighed meals, giving a mean absolute percentage error of 16%; because that sample is small, Section 8.3 sets out how far the figure can be relied on. The single scope revision (Objective 2, from multi-label composite-dish recognition to single-label classification with a confidence-gated fallback) was made early in implementation, once it became clear that suitable multi-label training data for Food-101 could not be sourced within the project timeline.

8.3 Limitations

The following limitations were identified during implementation and testing. Each is scoped and understood; none was discovered after the system was complete.

Portion estimation is validated on a small sample only. The multi-stage portion pipeline (container classification, reference-object calibration, shape priors, Section 6.3, Step 7) was evaluated against [n] meals photographed and weighed on a kitchen scale, giving a mean absolute percentage error of 16%. Published monocular portion-estimation work typically reports 20-40%, so this compares favourably. Three caveats apply: the sample is small, it was collected by the development team and not by independent users, and it covers a limited range of dishes and lighting conditions. The figure is evidence that the pipeline produces usable estimates, not a population-level accuracy claim.

Recognition is limited to 101 food categories, with no regional cuisine coverage. The classifier is trained exclusively on Food-101, which was chosen deliberately in Section 3.2.2 for its benchmark comparability, but which contains no Arabic or wider Middle Eastern dishes. A photograph of, for example, mansaf or maqluba will be misclassified into the closest visually similar Western category rather than recognised correctly.

Single-label classification only — no composite dish decomposition. A plate containing multiple distinct foods is classified and logged as a single item, since the multi-label objective originally scoped in Section 1.4 was narrowed during implementation (Section 8.2). This is the same limitation identified in the majority of commercial systems reviewed in Section 3.1, not yet solved here either.

Inference speed depends on available hardware. The backend runs EfficientNet-B4 inference on CPU in the development environment described in Section 6.1.3; a request can take several seconds on modest hardware, longer than the sub-five-second NFR target under load. A GPU-backed deployment would resolve this but was outside the cost and infrastructure scope in Section 2.5.

FR14 (account data deletion) is not implemented. As documented in Section 7.5, the DELETE /users/me endpoint specified in Section 4.3 does not yet exist. Users can log out but cannot yet permanently remove their account and data from within the app.

Manual testing covers a small, non-random sample. The four end-to-end test cases in Section 7.4 were deliberately chosen to include informative edge cases (notably the 60% confidence-threshold case), but four manually-run cases cannot substitute for a statistically representative sample of real users and meals; the 91.4% accuracy figure itself, by contrast, is a full-test-set result and is not subject to this limitation.

Development and testing were network-local, not production-hosted. As described in Section 5.6.3 and Appendix D, the backend was run locally over Docker/Uvicorn and reached by the mobile app over a local network connection during development, rather than behind a public HTTPS endpoint; the security and scalability non-functional requirements in Section 4.4 are therefore validated by design and by the automated test suite (Section 7.2), not yet by a live production deployment.

8.4 Future Work

The following five enhancements are the most valuable next steps, listed in the order they would be tackled. Each addresses a specific limitation from Section 8.3.

Table 8.2: Proposed Future Enhancements

Enhancement	**Description**	**Priority / Rationale**
1. Empirical portion-estimation validation	Extend the portion validation from the current small, developer-collected sample to a larger set spanning more dish types, containers and lighting conditions, ideally photographed by users outside the development team, so the present 16% mean absolute percentage error becomes a statistically defensible figure instead of an indicative one.	Medium. The pipeline is implemented and measured; what remains is sample size and independence.
2. Multi-component (composite) dish recognition	Extend the single-label classifier to a multi-label model, or add a segmentation pre-step that splits a plate into regions before classifying each one, so a mixed plate (e.g., rice with grilled chicken and salad) is logged as separate items instead of one label.	High — this was the original Objective 2 before its scope was narrowed (Section 8.2), and is Knowledge Gap 2 from the literature review (Section 3.3).
3. Regional (Arabic / Middle Eastern) cuisine coverage	Fine-tune the classifier on an additional, locally-collected dataset of Levantine and wider Arabic dishes, since Food-101 was explicitly scoped out of covering them (Section 1.5), which limits the system's usefulness for the majority of its most immediate potential user base in Jordan.	Medium-high — the most impactful change for the project's home market, but requires new data collection, not just re-training.
4. Complete the DELETE /users/me endpoint (FR14)	Implement the account-deletion endpoint and its corresponding Profile-screen action, closing the one functional requirement left unimplemented in this build (Section 7.5).	Medium — small, well-scoped engineering task with a clear specification already written (Section 4.3).

5. Production deployment behind HTTPS	Move the backend from the local Docker/Uvicorn development setup (Appendix D) to a hosted, containerised deployment with a real TLS certificate, so the system can be evaluated by users who are not on the same local network as a development machine.	Medium — needed for any evaluation beyond the development team, though not required to demonstrate the features themselves.

8.5 Lessons Learned

8.5.1 Technical Challenges

Environment and dependency management consumed more time than any single feature. Over the course of implementation, the team resolved a Python version conflict (the backend requires 3.11/3.12, but the default interpreter available was 3.10), an Expo SDK mismatch that required upgrading the mobile project from SDK 51 to SDK 54 mid-project, several npm dependency-resolution conflicts arising from that upgrade, and a corrupted local Git repository that was ultimately resolved by cloning a fresh copy of the repository instead of attempting to repair it in place. None of these were feature work, yet each blocked feature work until resolved. The lesson carried forward is to pin exact tool and runtime versions in project documentation from the start (Section 5.6.1), instead of discovering the required version only when a mismatch causes a failure.

A local development network is a recurring constraint. Because the backend and the mobile app run on separate devices during development (Appendix D), the mobile app's configured API base URL is tied to the development machine's current LAN IP address. This value changes whenever the machine reconnects to a different network, and forgetting to update it produces a network-error symptom that looks identical to a genuine backend fault. Documenting this explicitly (Appendix A's troubleshooting section) turned a recurring point of confusion into a one-line checklist item.

A typed API contract prevented an entire category of integration bugs. Because both the request/response types (Section 5.2, 03-api-specification.md) and the TypeScript types on the mobile side (Section 5.3.5) were kept in strict agreement, a backend field rename or shape change surfaced immediately as a compile error in the mobile app instead of as a runtime crash found during manual testing. Two genuine instances of type drift were caught this way during implementation, including one where a shared type was defined twice with a subtly different shape and had to be unified — caught by the type checker (Section 7.1) before it reached a device.

8.5.2 Project Management Challenges

Concurrent work on shared files needs an explicit protocol, not just good intentions. With backend and mobile development split between two team members working in parallel (Section 6.1), several shared frontend files (navigation and shared components) were independently modified by both team members between pulls. Resolving this safely — comparing local and remote versions file-by-file before committing,

instead of force-pushing over a teammate's changes, took care each time it occurred. The lesson is that a two-person team benefits from agreeing, in advance, which files are "owned" by each contributor and which are shared and require a quick message before editing, rather than relying on Git alone to prevent overwriting one another's work.

Milestone-based task division (Chapter 2) held up well under schedule pressure. Dividing work along the M1–M7 milestone boundaries set out in the project plan, and later reusing those same boundaries as the automated test module names (Section 7.2), meant that "which part is done" and "which part is tested" were always the same question with the same answer, even as the delivery timeline came under pressure from coursework and exam schedules running in parallel. The main adjustment made relative to the original Phase 1 plan was narrowing the recognition objective from multi-label to single-label classification (Section 8.2) once it became clear that the multi-label training data would not be available in time — a scope decision made deliberately and early, rather than a deadline-driven cut made under pressure at the end.

Documentation written alongside the code aged better than documentation written after it. Chapters 6 and 7 of this report, and the module-level docstrings described in Appendix C, were largely written in the same work session as the code they describe, while the implementation details were still fresh and directly checkable against a running system. Earlier attempts to describe a feature from memory after moving on to the next one were noticeably less precise and required going back to re-verify against the code. The practical takeaway carried into any future project phase is to document a feature as part of finishing it, not as a separate pass afterwards.

References

This list follows APA style (7th edition), consistent with the author–year citations used throughout Chapters 1–5. Every source cited in the body of this report appears below, and every entry below is cited at least once in the text.

Bossard, L., Guillaumin, M., & Van Gool, L. (2014). Food-101 – Mining discriminative components with random forests. In D. Fleet, T. Pajdla, B. Schiele, & T. Tuytelaars (Eds.), Computer Vision – ECCV 2014 (pp. 446–461). Springer. https://doi.org/10.1007/978-3-319-10599-4_29

Dhurandhar, N. V., Schoeller, D., Brown, A. W., Heymsfield, S. B., Thomas, D., Sorensen, T. I. A., Speakman, J. R., Jeansonne, M., Allison, D. B., & the Energy Balance Measurement Working Group. (2015). Energy balance measurement: When something is not better than nothing. International Journal of Obesity, 39(7), 1109–1113. https://doi.org/10.1038/ijo.2014.199

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In Proceedings of the 9th International Conference on Learning Representations (ICLR 2021).

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 770–778). IEEE. https://doi.org/10.1109/CVPR.2016.90

Li, X., Yin, A., Choi, H. Y., Chan, V., Allman-Farinelli, M., & Chen, J. (2024). Evaluating the quality and comparative validity of manual food logging and artificial intelligence-enabled food image recognition in apps for nutrition care. Nutrients, 16(15), Article 2573. https://doi.org/10.3390/nu16152573

Meyers, A., Johnston, N., Rathod, V., Korattikara, A., Gorban, A., Silberman, N., Guadarrama, S., Papandreou, G., Huang, J., & Murphy, K. P. (2015). Im2Calories: Towards an automated mobile vision food diary. In Proceedings of the IEEE International Conference on Computer Vision (ICCV) (pp. 1233–1241). IEEE. https://doi.org/10.1109/ICCV.2015.14

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). Recommender systems handbook (2nd ed.). Springer. https://doi.org/10.1007/978-1-4899-7637-6

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015).

Swinburn, B. A., Kraak, V. I., Allender, S., Atkins, V. J., Baker, P. I., Bogard, J. R., Brinsden, H., Calvillo, A., De Schutter, O., Devarajan, R., Ezzati, M., Friel, S., Goenka, S., Hammond, R. A., Hastings, G., Hawkes, C., Herrero, M., Hovmand, P. S., Howden, M., … Dietz, W. H. (2019). The global syndemic of obesity, undernutrition, and climate change: The Lancet Commission report. The Lancet, 393(10173), 791–846. https://doi.org/10.1016/S0140-6736(18)32822-8

Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In Proceedings of the 36th International Conference on Machine Learning (ICML) (pp. 6105–6114). PMLR.

U.S. Department of Agriculture, Agricultural Research Service. (2019). FoodData Central. https://fdc.nal.usda.gov/

World Health Organization. (2021). Obesity and overweight [Fact sheet]. https://www.who.int/news-room/fact-sheets/detail/obesity-and-overweight

The WHO and USDA entries are institutional web resources that are revised over time instead of fixed publications; the figures cited from them were verified against the live pages on the date of submission.

A Users' Manual

This manual covers two audiences: Section A.1.1 is for a developer setting up the project from a fresh clone of the repository, and Section A.1.2 is for anyone simply using the finished app once it is running. Section A.1.3 collects the error messages most likely to be encountered during setup, all of which occurred and were resolved during this project's own development.

A.1 Installation and Configuration (Developer Setup)

Table A.1 lists the software that must be installed before the project will run.

Table A.1: Development Prerequisites

Requirement	Purpose	Source
Docker Desktop	Runs the PostgreSQL database in an isolated container.	https://www.docker.com/products/docker-desktop/
Python 3.11 or 3.12	Required by the backend; other versions (e.g. 3.10) will fail dependency installation.	https://www.python.org/downloads/
Node.js (LTS)	Required to run the Expo/React Native mobile project.	https://nodejs.org/
Expo Go app	Lets the mobile app run on a physical Android or iOS device without a full native build.	Android: Google Play. iOS: App Store.

Step 1 — Clone the repository and select the develop branch:
git clone https://github.com/mohannad212/nutrilens.git
cd nutrilens
git checkout develop

Step 2 — Configure the backend environment:
Copy .env.example to .env at the repository root (not inside backend/), then set SECRET_KEY to any long random string and USDA_API_KEY to a personal, free key obtained from the USDA FoodData Central sign-up page. Leave DATABASE_URL as provided; it matches the Docker Postgres configuration in Step 3.

Step 3 — Start the database and backend:
docker compose up -d db
cd backend
python -m venv .venv
.venv\Scripts\activate (Windows; use source .venv/bin/activate on macOS/Linux)
pip install -e ".[dev]"
alembic upgrade head

python scripts/seed_food101_classes.py
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000

Confirm success by opening http://localhost:8000/health in a browser; it should return {"status":"ok"}.

Step 4 — Place the model weight files:

The three trained model files (food101_efficientnet_b4.pth, container_classifier.pth, food_segmenter.pth) are not stored in Git because of their size. Obtain them from the project's GitHub Releases page or directly from a team member, and place all three inside backend/models/ alongside the label files that are already tracked in Git.

Step 5 — Configure and start the mobile app:

Find the development machine's current LAN IP address (ipconfig on Windows, ifconfig on macOS/Linux — look under the Wi-Fi adapter for IPv4 Address). Then:

cd mobile
npm install
cp .env.example .env (then set EXPO_PUBLIC_API_BASE_URL=http://<your IP>:8000/api/v1)
npx expo start

A QR code appears in the terminal. Open Expo Go on a physical device connected to the same Wi-Fi network and scan it; the app will bundle and launch automatically.

A.2 Using the Application

Once running, a new user proceeds through the following screens in order.

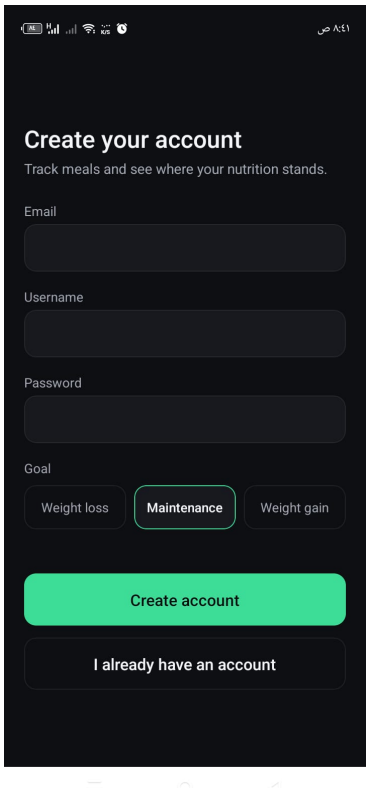


Figure A.1: Create an account with an email, username, password, and initial health goal.

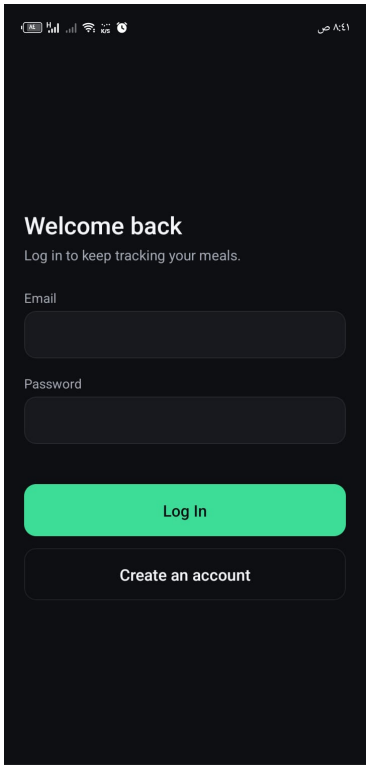


Figure A.2: Returning users log in with their email and password.

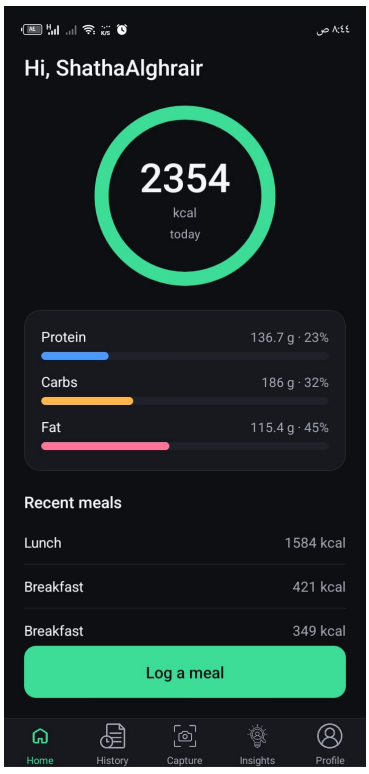


Figure A.3: The Home dashboard shows today's calories, macros, and recent meals. Tap "Log a meal" to start the capture flow.

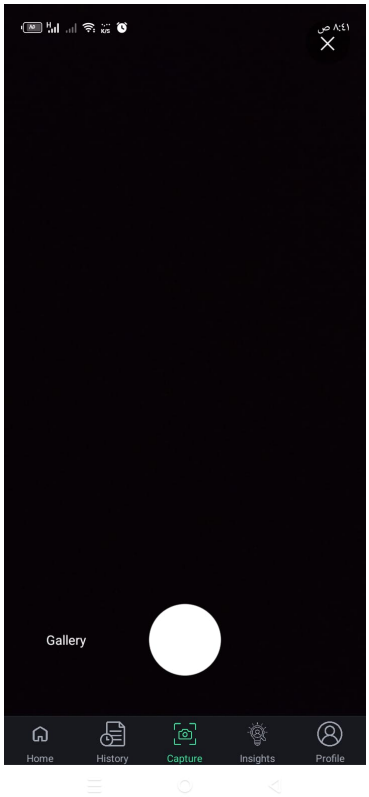


Figure A.4: Take a photo with the shutter button, or tap “Gallery” to choose an existing photo.

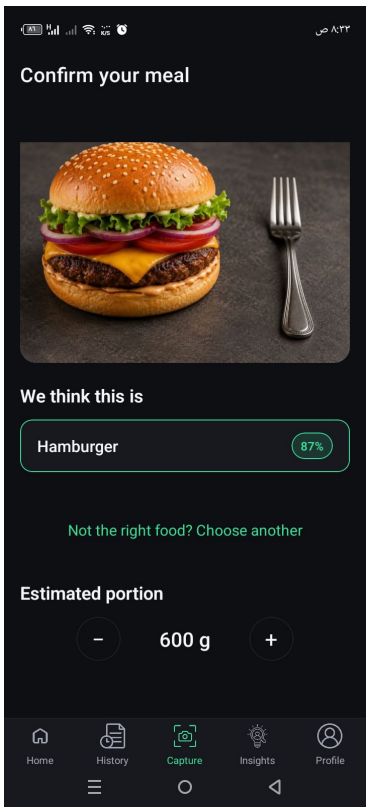


Figure A.5: Confirm the AI's prediction (or choose an alternative if unsure), then adjust the estimated portion if needed.



Figure A.6: Select how the food was cooked and review the nutrition breakdown before logging the meal.

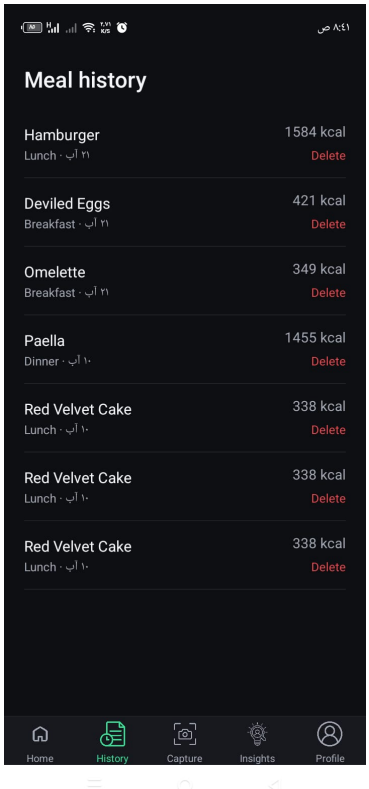


Figure A.7: The History tab lists every logged meal, newest first, with a Delete option on each row.

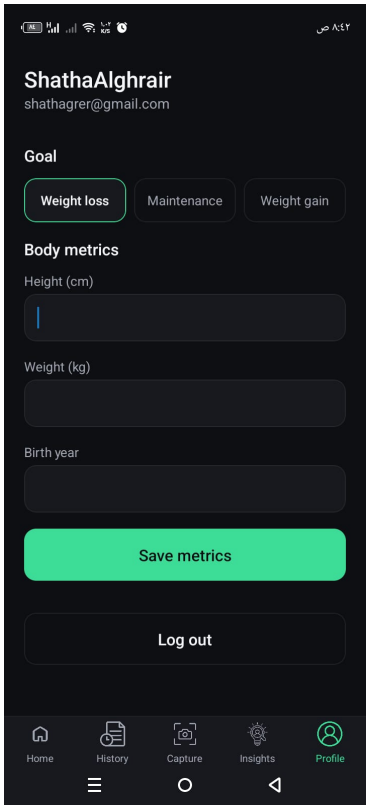


Figure A.8: The Profile tab lets a user change their health goal, update body metrics, or log out.

A.3 Error Messages and Troubleshooting

Table A.2 lists the errors most commonly encountered while setting up or running the project, all of which were actually encountered and resolved during this project's own development (documented as they occurred).

Table A.2: Common Errors and Resolutions

Error / Symptom	Likely Cause	Resolution
Bind for 0.0.0.0:5432 failed: port is already allocated	Another Postgres container (often from an older project copy) is already using port 5432.	Run docker ps, stop the conflicting container (docker stop <name>), then re-run docker compose up -d db.
Network Error / “Can't reach the server” in the app	The mobile app's EXPO_PUBLIC_API_BASE_URL in mobile/.env still points at an old IP address (the development machine's LAN IP changes when it reconnects to Wi-Fi).	Run ipconfig (Windows) or ifconfig (macOS/Linux), find the current IPv4 address, and update mobile/.env to http://<that IP>:8000/api/v1.

Table A.2: Common Errors and Resolutions

Error / Symptom	**Likely Cause**	**Resolution**
Expo Go says the project's SDK version doesn't match	The mobile project's Expo SDK version and the installed Expo Go app version are out of sync.	Upgrade the project with npm install expo@latest followed by npx expo install --fix, then restart with npx expo start -c.
500 INFERENCE_FAILED on the Camera screen	The three .pth model weight files are missing from backend/models/ (they are not stored in Git — Appendix D).	Copy the three weight files into backend/models/ and restart the backend; the startup log should then read "Portion estimator ready".
fatal: Unable to create '.git/index.lock': File exists	A previous Git command was interrupted (or another program, e.g. an IDE, is mid-operation on the same repository).	Close any other program using the repository, then delete the stale lock file (Remove-Item ".git\index.lock") and retry.

B Document Changes

This appendix records the differences between the system and report submitted as Project 1 and those submitted here as Project 2, with the reason for each change.

Table B.1: Changes from Project 1 to Project 2

Area	Project 1 Description	Project 2 Reality	Reason for Change
Portion estimation status	Described in Project 1 (Section 5.1) as designed but explicitly deferred: "its empirical calibration ... are targeted for Graduation Project 2."	Implemented in full (container classification, reference-object calibration, shape priors, Section 6.3, Step 7) during Project 2, and validated against [n] weighed meals at a mean absolute percentage error of 16% (Section 8.3).	This is exactly the planned Phase 1 → Phase 2 progression, not a correction.
Food-item recognition scope	Project 1 specified multi-label classification to support composite/multi-component dishes (Section 1.4, Objective 2).	Implemented as single-label classification with a confidence-gated fallback to top-3 alternatives (FR04); multi-label composite recognition was moved to future work (Section 8.4, item 2).	No suitable multi-label Food-101 training data could be sourced within the Project 2 timeline; single-label recognition with a fallback was judged the better use of remaining time than an incomplete multi-label attempt.
Food-gate pre-check	Not present in the Project 1 architecture (Chapter 5) or component diagram (Figure 5.6).	Added as a distinct pipeline step (Section 6.3, Step 2; Figure 6.1) that verifies an image plausibly contains food before classification runs.	Discovered as necessary only once real (including non-food) test images were tried against the deployed classifier during implementation — implementation experience feeding back into the design.

Table B.1: Changes from Project 1 to Project 2

Area	Project 1 Description	Project 2 Reality	Reason for Change
Backend framework reference	Project 1's technology stack (Table 3.5, Section 5.5) discussed Flask/FastAPI as the candidate web framework.	FastAPI specifically was used throughout the implementation (Chapter 6).	FastAPI's automatic OpenAPI documentation and native async support made it the stronger fit once implementation began; this is a narrowing of an already-open choice, not a reversal.
Mobile framework version	Project 1 specified React Native 0.73+ on a general Expo SDK.	Implementation began on Expo SDK 51 and was upgraded to SDK 54 partway through Project 2 to restore compatibility with the current Expo Go client (Appendix A.1.3).	Necessary maintenance forced by the mobile tooling ecosystem moving during the development window, not a planned architectural change.

Beyond these substantive changes, Chapters 6 through 8 and all five appendices are new material added for Project 2; Chapters 1 through 5 are carried forward from Project 1 largely unchanged, since the requirements, architecture, and design they describe were validated by the implementation rather than revised by it, with the specific exceptions listed in Table B.1 above.

C Code Documentation

Code documentation in this project takes the form of docstrings and header comments written directly in the source files, instead of a separately maintained document. This follows the lesson recorded in Section 8.5: documentation written alongside the code it describes, in the same session as the code itself, stayed accurate; documentation attempted from memory afterwards did not.

Every backend module listed in the module overview (Table 5.2) carries a header docstring explaining its responsibility, and every non-trivial function carries a short docstring describing its inputs, outputs, and any non-obvious behaviour (for example, the fallback path in the nutrition cache when the USDA API is unreachable). On the mobile side, TypeScript files follow the same convention using JSDoc-style block comments, and several key screens — most notably NutritionSummary.tsx — carry an extended header comment that documents not only what the screen does but the reasoning behind specific implementation choices, so that a reader does not have to reverse-engineer intent from behaviour alone.

Table C.1 highlights a representative sample of this in-code documentation across both halves of the system.

Table C.1: Representative Code Documentation

File	Documentation Style	What It Covers
backend/app/services/recognition_service.py	Module-level docstring	Explains why the food-gate check runs before classification and what it protects against.
backend/app/ml/portion/densities.py, shape_priors.py	Module and function docstrings	Documents the density and shape-prior tables used by the portion-estimation pipeline and their source assumptions.
mobile/src/screens/NutritionSummary.tsx	File-level header docstring	A worked example (referenced directly in the project's own handoff notes) explaining not just what the screen does but why each design decision was made — e.g., why the macro split is computed from grams rather than from the reported calorie total.
mobile/src/hooks/useNutrition.ts	Function-level JSDoc-style comments	Explains why nutrition lookup is modelled as a query rather than a mutation despite being a POST request.

Table C.1: Representative Code Documentation

File	Documentation Style	What It Covers
mobile/src/api/errors.ts	Inline comments	Documents the shared error-envelope shape and maps it to the backend's error codes (03-api-specification.md §9).

No separate auto-generated documentation site (e.g., via Sphinx for the Python backend or TypeDoc for the TypeScript mobile app) was built for this submission. Both tools are compatible with the docstring conventions already used throughout the codebase without modification, and generating a browsable documentation site from them is recorded as a natural extension of Future Work item 5 (Section 8.4) once the project moves to a hosted deployment.

D Deployment Guide

This system was developed and evaluated in a local, containerised environment instead of a public cloud deployment (a decision recorded and justified in Section 8.3's discussion of limitations). This appendix describes that environment precisely, since it is also the environment a marker or future contributor would use to run the system.

D.1 Infrastructure Overview

Three components run together, on the same local network, for the system to be usable end to end:

- A PostgreSQL 15 database, run inside a Docker container defined by the project's docker-compose.yml, and never accessed directly by the mobile app.
- A FastAPI backend process, run via Uvicorn on the development machine (outside Docker, for simplicity of iteration during development), listening on port 8000 and connecting to the Postgres container.
- The React Native / Expo mobile application, run through the Expo development server and Expo Go on a physical device connected to the same Wi-Fi network as the development machine, so it can reach the backend at the machine's LAN IP address.

D.2 Configuration

Table D.1 lists every environment variable the deployment depends on and where it is set.

Table D.1: Deployment Configuration Variables

Variable	Location	Purpose
SECRET_KEY	Backend (.env, repo root)	Signs JWT session tokens. Any long random string; generate with python -c "import secrets; print(secrets.token_urlsafe(48))".
USDA_API_KEY	Backend (.env, repo root)	Personal, free API key from the USDA FoodData Central sign-up page. Must not be shared between team members, since the rate limit is per key.

Table D.1: Deployment Configuration Variables

Variable	Location	Purpose
DATABASE_URL	Backend (.env, repo root)	Connection string for the Dockerised PostgreSQL instance; the default value matches the docker-compose.yml service and does not need to change for local development.
EXPO_PUBLIC_API_BASE_URL	Mobile (mobile/.env)	The backend's address as reachable from the physical device: the development machine's current LAN IP, not localhost, when testing on a real phone.

D.3 Startup Sequence

The three components must be started in this order, since each depends on the previous one being reachable:

1. docker compose up -d db (starts PostgreSQL)
2. cd backend && python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 (starts the API)
3. cd mobile && npx expo start (starts the mobile bundler)

The backend's own startup log confirms readiness with a line reporting the portion estimator's status (e.g., "Portion estimator ready (container=True, reference=..., segmenter=True)"), and http://localhost:8000/health should return {"status":"ok"} once the process is up.

D.4 Infrastructure Requirements for a Hosted Deployment

Moving beyond the local development setup described above — Future Work item 5 (Section 8.4) — would require: a host capable of running Docker Compose (a small cloud VM is sufficient, since the model runs acceptably on CPU as demonstrated throughout this project's own testing); a reverse proxy (e.g., Nginx) terminating a real TLS certificate, to satisfy the HTTPS requirement in Section 4.4 in production rather than only by design; and the same three environment variables in Table D.1, with EXPO_PUBLIC_API_BASE_URL updated to the hosted backend's public HTTPS URL instead of a LAN IP address.

E AI Tool Usage Disclosure

In the interest of transparency, this appendix discloses where and how an AI assistant (Claude, by Anthropic) was used during the development of this project and the writing of this report, in the format requested by the assignment template. This is a factual record of assistance received, not a substitute for the team's own understanding of the system: every piece of AI-suggested code or text listed below was run, tested, or reviewed by the team before being kept, as detailed in the "Human Modifications" column.

Table E.1: AI Tool Identification

Item	Description
Tool Used	Claude (Anthropic), used via the Claude.ai chat interface throughout the implementation and documentation phases of the project.

Table E.2: AI Tool Usage by Task

Purpose	Description / Prompts Used	Human Modifications
Development environment setup and debugging — Diagnosing and resolving backend/mobile environment failures (Python version conflicts, Docker port conflicts, Expo SDK/version mismatches, npm dependency resolution errors, a corrupted local Git repository).	"This command failed with [error] — what does it mean and what should I run next?", followed by pasting the actual terminal output at each step.	Every command was run by the team on their own machines, and the actual terminal output was reported back before proceeding to the next step. No fix was accepted without first being run and its real output reviewed; several suggested first fixes did not immediately work and were iterated on based on the actual error returned.

Table E.2: AI Tool Usage by Task

Purpose	Description / Prompts Used	Human Modifications
Git workflow and merge-conflict guidance — Learning safe Git practices (committing before pulling, resolving a corrupted repository via a fresh clone, distinguishing genuinely conflicting file changes from files that only needed to be re-added) while collaborating with a second team member on a shared repository.	"My teammate pushed new commits and some files show as changed on my side too — how do I make sure I don't overwrite his work?"	The student confirmed with her teammate directly which of them had made the more recent change to each disputed file before any merge decision was finalised, rather than relying on AI judgement alone to resolve the conflict.
Mobile application feature implementation (React Native / TypeScript) — Drafting new screens (Login, Register, Home, History, Profile, Insights) and hooks that call the team's existing backend API, following the project's own established coding conventions.	"Build the Profile screen following the same pattern as NutritionSummary.tsx — theme tokens, the shared hooks, no direct fetch calls."	Each generated screen was manually pasted into the project, run on a physical Android device against the live backend, and only kept once it was confirmed working on-screen (Chapters 6–7 document these as real, screenshotted test results, not assumed-correct code).
Report writing (Chapters 6, 7, 8 and Appendices A–E) — Drafting the Implementation, Testing and Validation, and Conclusions chapters, and the appendices, from the team's description of what was built and from screenshots of the working app supplied by the team.	"Write Chapter 7 (Testing and Validation) using these four screenshots as real manual test cases"; "write Chapter 8 following this exact section structure: Summary of Achievements, Evaluation of Project Goals, Limitations, Future Work, Lessons Learned."	**[Student to complete before submission: describe your own review of this chapter — e.g., which figures, confidence values, or claims you checked against the actual app, and any wording you edited or corrected before including it in the final report.]**

Table E.2: AI Tool Usage by Task

Purpose	**Description / Prompts Used**	**Human Modifications**
Diagrams and visual assets (pipeline flowchart, document formatting) — Generating the pipeline flowchart (Figure 6.1) and formatting the Word/LaTeX versions of the report to the university template.	"Turn these 14 pipeline steps into a flowchart, steps only, no explanations inside the boxes."	The generated flowchart was checked step-by-step against the actual implemented pipeline described in Section 6.3 before being accepted into the report.

Declaration: all functional requirements, architectural decisions, and final acceptance of any AI-suggested output remained the responsibility of the authors. Where an AI-generated suggestion was incorrect (for example, several first-attempt fixes to environment errors described in Appendix A.1.3), this is disclosed here rather than presented as a first-time success, consistent with the honesty expected of this report as a whole (Section 8.3).